Stage II : Project Report on

Idea Collision Generator

A Personalized Knowledge-Intelligence Engine

submitted in the partial fulfillment of the requirement of Degree in Bachelor of Technology in Information Technology

by

Ruchi Ingale (231081904)
Vaibhav Bhagwat (231080901)
Vaibhavi Chaughule (231081907)
Omkar Kambre (231080903)

Under the Guidance Of

Prof. Shrinivas Khedkar

Department of Information Technology
Veermata Jijabai Technological Institute, Mumbai - 400019
(Autonomous Institute Affiliated to University of Mumbai)

2025 – 2026

Veermata Jijabai Technological Institute, Matunga

Department of Information Technology

CERTIFICATE

This is to certify that the Annual Progress Stage-II Report entitled

Idea Collision Generator – A Personalized Knowledge-Intelligence Engine

by

Ruchi Ingale (231081904)
Vaibhav Bhagwat (231080901)
Vaibhavi Chaughule (231081907)
Omkar Kambre (231080903)

is in partial fulfillment for the degree of B.Tech in Information Technology.

Examiner 1	Examiner 2	Examiner 3

Guide
Prof. Shrinivas Khedkar

Declaration

We hereby declare that the work presented in this report entitled **"Idea Collision Generator – A Personalized Knowledge-Intelligence Engine"** is original and has been carried out by the authors under the guidance of Prof. Shrinivas Khedkar. To the best of our knowledge, this report does not contain any material previously published or written by another person, except where due references are made.

Ruchi Ingale	Vaibhav Bhagwat
Vaibhavi Chaughule	Omkar Kambre

Date: 25/11/2025
Place: VJTI, Mumbai

Acknowledgement

We would like to express our sincere gratitude to Prof. Shrinivas Khedkar for his invaluable guidance, encouragement, and support throughout the course of this project titled Idea Collision Generator His expertise, timely feedback, and insightful suggestions played a crucial role in shaping the direction and quality of our work. We are equally grateful to the Department of Computer Engineering and Information Technology , Veermata Jijabai Technological Institute (VJTI), for providing us with the necessary academic environment, resources, and infrastructure to successfully carry out this project. The constant support from the faculty and staff greatly contributed to our learning experience. We would also like to acknowledge the collaborative efforts and contributions of all team members of Group 2520, whose dedication, cooperation, and shared commitment made this project possible. Finally, we extend our heartfelt appreciation to our families and friends for their continuous moti vation and understanding throughout the duration of this work.

Finally, we extend our heartfelt appreciation to our families and friends for their continuous motivation and understanding throughout the duration of this work.

Ruchi Ingale (231081904)
Vaibhav Bhagwat (231080901)
Vaibhavi Chaughule (231081907)
Omkar Kambre (231080903)

Fourth Year, B.Tech. Computer Engineering & Information Technology
Department of Computer Engineering & Information Technology
Veermata Jijabai Technological Institute, Mumbai 400019

Date: 25/11/2025

Place: VJTI, Mumbai

Abstract

The Idea Collision Generator (ICD) is a fully deployed, personalized knowledge-intelligence system designed to support creative reasoning, interdisciplinary research, and exploratory thinking within the Technology domain. In an era of pervasive information overload, researchers, engineers, and students consume vast amounts of digital content but routinely fail to surface non-obvious connections across specialized subdomains—a phenomenon that reinforces knowledge silos and impedes innovation. Traditional tools such as bookmarks and note-taking applications organise information linearly but offer no mechanism for semantic cross-domain discovery or proactive insight synthesis.

This report presents the complete design, implementation, and experimental evaluation of the ICD system, which has been successfully deployed as an end-to-end, browser-native pipeline. The system is built upon an 8-subdomain Technology taxonomy spanning Artificial Intelligence, Data Science and Analytics, Software Engineering, Cybersecurity, Networking and Cloud, Hardware and Robotics, Human-Computer Interaction, and Emerging Technologies. A multi-stage NLP extraction pipeline combining spaCy Named Entity Recognition, noun phrase chunking, aggressive relevance filtering, and fuzzy taxonomy matching converts raw web articles into structured, subdomain-classified concept nodes. These nodes are persisted in a Neo4j knowledge graph, with relational metadata stored in PostgreSQL and the complete service stack containerised using Docker Compose.

A Cross-Subdomain Collision Engine identifies concept pairs from structurally distant subdomains and invokes Google Gemini 1.5 Flash to synthesise structured research insights comprising an executive summary, technical convergence mechanism, market validity assessment, and implementation challenges. The system is delivered through a Chrome Manifest V3 extension for real-time article capture, a FastAPI backend for ingestion and orchestration, and a Next.js research dashboard featuring interactive 3D knowledge graph visualisation powered by Three.js.

Experimental evaluation across five end-to-end test phases and multiple ingested articles demonstrates that the hybrid Knowledge Graph and Large Language Model approach outperforms individual pipeline configurations: the combined KG+LLM method achieves a Relevance Score of 0.81, a Novelty Score of 0.76, and an NER F1 of 0.87, compared to 0.74 relevance and 0.58 novelty for the LLM-only baseline. The enhanced multi-stage NLP pipeline reduces spurious node creation by approximately 35% under noisy input conditions. All five deployment phases—infrastructure, backend, frontend, browser extension, and end-to-end testing—have been verified and confirmed operational.

Keywords: Knowledge Graphs, Natural Language Processing, Large Language Models,

Cross-Domain Insights, Gemini 1.5 Flash, Idea Collision, Personalized Knowledge Engine, spaCy, Neo4j, Chrome Extension, FastAPI, Next.js.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background and Context

The accelerating pace of technological advancement has created an unprecedented information landscape. Researchers and engineers today navigate publications, blog posts, technical documentation, and news articles spanning dozens of interconnected subdomains—from Artificial Intelligence and Cybersecurity to Quantum Computing and Human-Computer Interaction. While the volume of accessible knowledge has grown exponentially, the cognitive capacity of any individual to retain, connect, and synthesise ideas across these boundaries has remained largely fixed.

This asymmetry between available information and human cognitive bandwidth gives rise to a well-documented phenomenon: *knowledge silos*. When a researcher primarily consumes content within a single subdomain, their mental model of adjacent fields atrophies. The cross-pollination of concepts—a recognised driver of breakthrough innovation—occurs infrequently and largely by chance. A cybersecurity researcher who never encounters developments in Human-Computer Interaction may miss the insight that zero-knowledge proof protocols could reshape privacy-preserving interface design. A data scientist unfamiliar with robotics literature may overlook the relevance of swarm intelligence algorithms to distributed machine learning optimisation.

Traditional knowledge management tools compound this problem rather than solving it. Bookmarking services preserve URLs but offer no semantic structure. Note-taking applications such as Notion or Obsidian require significant manual curation effort and surface connections only within the user's own explicit notes. Search engines are reactive and query-driven: they retrieve information the user already knows to ask for, rather than proactively surfacing hidden relationships between concepts the user has passively encountered.

The emergence of Knowledge Graphs (KGs) and Large Language Models (LLMs) has created a new design space for intelligent knowledge systems. Knowledge graphs provide structured, queryable representations of entities and their relationships, enabling efficient traversal across conceptual boundaries. LLMs provide generative reasoning capabilities, transforming structured concept pairs into coherent, contextualised insights. The convergence of these two technologies—a pattern identified as the "Synergized Framework" paradigm in recent surveys **ibrahim2024survey**—offers a promising foundation for building systems that actively

bridge knowledge silos.

1.2 Problem Statement

Despite the availability of advanced NLP and graph database technologies, no existing browser-native system addresses all of the following requirements simultaneously:

1. **Passive, real-time knowledge capture**: Automatically tracking what a user reads without requiring manual input or annotation.
2. **Structured concept extraction**: Converting unstructured web text into semantically meaningful, subdomain-classified concept nodes.
3. **Personalised, evolving knowledge graph**: Maintaining a dynamic graph that grows with the user's reading history rather than operating on a static corpus.
4. **Cross-domain collision synthesis**: Intentionally identifying concept pairs from structurally distant subdomains and generating actionable interdisciplinary insights.
5. **Accessible, interactive delivery**: Presenting generated insights through an intuitive, visually engaging interface that supports graph exploration.

Existing approaches address subsets of these requirements in isolation. Literature-based discovery systems **wang2023learning** operate on static academic corpora and require seed terms. Recommendation systems **polleti2025generating** optimise for relevance within a domain rather than serendipitous cross-domain discovery. Web augmentation tools **wischenbart2021enga** enable browser-side personalisation but lack semantic depth. The Idea Collision Generator is designed to satisfy all five requirements within a single, unified, deployable system.

1.3 Motivation

The Idea Collision Generator is motivated by the concept of *bisociation*, introduced by Arthur Koestler to describe the creative act of connecting two previously unrelated frames of reference to produce a novel insight. Bisociative thinking underlies many significant innovations: the connection between statistical mechanics and information theory produced Shannon entropy; the connection between evolutionary biology and computer science produced genetic algorithms; the connection between neuroscience and machine learning produced backpropagation.

The ICD system operationalises bisociation computationally. By constructing a personalised knowledge graph from a user's reading history and then deliberately selecting concept pairs that are maximally structurally distant in that graph, the system creates the conditions for bisociative insight to emerge—amplified by the generative reasoning capabilities of a large language model.

The specific motivations driving design decisions include:

- **Combat information overload**: Structure and semantically index what users read, transforming passive consumption into active knowledge accumulation.
- **Bridge knowledge silos**: Automatically surface cross-subdomain connections that a user would be unlikely to encounter through normal reading patterns.
- **Generate actionable insights**: Produce structured research outputs—not merely concept associations—that include technical mechanisms, feasibility assessments, and market potential analyses.
- **Preserve reading context**: Ground all generated insights in the user's actual reading history, ensuring outputs are relevant to their real interests and background.
- **Enable serendipity by design**: Inject "fringe" concepts from the periphery of the knowledge graph to foster high-novelty "Black Swan" ideas that would not emerge from similarity-based recommendation alone.

1.4 Objectives

The primary objectives of this project, all of which have been achieved in the final system, are:

1. **Browser Extension**: Design and deploy a Chrome Manifest V3 extension that passively captures readable content from any web page and transmits structured payloads to the backend in real time.
2. **NLP Pipeline**: Implement a multi-stage concept extraction pipeline combining Named Entity Recognition, noun phrase chunking, relevance filtering, and subdomain classification using a predefined 8-subdomain Technology taxonomy.
3. **Knowledge Graph**: Construct and maintain a personal knowledge graph in Neo4j that stores concept nodes with subdomain metadata and `MENTIONS` relationship edges to source articles, enabling efficient cross-subdomain querying.
4. **Collision Engine**: Engineer a Cross-Subdomain Collision Engine that selects concept pairs from structurally distant subdomains and invokes Google Gemini 1.5 Flash to synthesise structured, multi-section research reports.
5. **Frontend Dashboard**: Deliver a Next.js research dashboard with real-time statistics, a 3D interactive knowledge graph visualisation, a collision explorer, and article management—all served from verified API endpoints.
6. **End-to-End Deployment**: Containerise all infrastructure services (Neo4j, Redis, PostgreSQL) using Docker Compose and verify the complete data flow from browser extension capture through to collision display in the dashboard.

1.5 Scope and Delimitations

The system operates within the Technology domain exclusively, leveraging an 8-subdomain taxonomy to ensure high-precision concept classification. While the underlying architecture is domain-agnostic and could in principle be extended to other fields (medicine, economics, humanities), the current implementation is scoped to Technology content to ensure classification quality. The system processes English-language content; multilingual support is identified as a future enhancement. The system operates as a local deployment (localhost) in its current form; cloud deployment and user authentication are listed as planned future improvements. The collision engine generates pairwise insights; multi-concept collisions involving three or more nodes are left for future work.

1.6 Report Organisation

This report is structured as follows. Chapter 2 presents a systematic literature review covering knowledge graph construction, web content extraction, explainable recommendations, visual feature-based extraction, and LLM-KG integration paradigms, concluding with an analysis of research gaps that motivate the ICD system. Chapter 3 describes the complete system design and architecture, including the technology taxonomy, module descriptions, knowledge graph schema, and data flow. Chapter 4 details the methodology, covering the NLP extraction pipeline, collision engine algorithm, Gemini prompt engineering, and performance evaluation metrics. Chapter 5 presents the full implementation, including technology stack, API endpoints, Docker configuration, code structure, and the browser extension. Chapter 6 reports experimental results and discussion. Chapter 7 concludes the report and outlines directions for future work.

Chapter 2

Literature Survey

2.1 Overview

Knowledge graph construction, natural language processing for information extraction, web content extraction, recommender systems, and Large Language Model integration have each been studied extensively in isolation. However, their convergence into a unified, browser-native, personalised intelligence engine that actively supports creative ideation across technical subdomains represents an emerging and significantly underexplored research direction. This chapter reviews five foundational works that collectively inform the design, architecture, and evaluation of the Idea Collision Generator, followed by a synthesis of research gaps that motivate the proposed system.

2.2 Knowledge Graph Construction from Unstructured Text

Iqra Muhammad et al. iqra2020open proposed an Open Information Extraction (OIE) approach for constructing literature knowledge graphs from scholarly documents. Their method employed RnnOIE, a deep learning-based model, to extract relational triples in the form $\langle \textit{subject}, \textit{relation}, \textit{object} \rangle$ from unstructured text. The system processes documents through three stages: (1) triple extraction using recurrent neural models, (2) triple filtering using noun chunking and frequency-based lexicons, and (3) concept linking to domain-specific ontologies (UMLS for the medical domain).

To address knowledge graph incompleteness, the authors utilised knowledge embeddings to predict missing links, defining a plausibility score function:

$$f_r(h, t \mid \Theta) = \mathbf{h}^\top \mathbf{M}_r \mathbf{t} \tag{2.1}$$

where $\mathbf{h}$ and $\mathbf{t}$ are entity embedding vectors, $\mathbf{M}_r$ is the relation-specific transformation matrix, and Θ denotes the set of learned parameters. This score measures the likelihood of a valid connection between head entity h and tail entity t under relation r.

The approach demonstrated an F-score of 51% on a clinical research methodology dataset and 37% on standard benchmark datasets. However, the method faced several limitations:

low coverage (42% using the graph-only approach without embedding-based completion), degraded performance on non-English content due to reliance on English-trained NER models, and inability to handle the dynamic, continuously evolving nature of personalised knowledge graphs where new content is ingested in real time.

Relevance to ICD: The ICD system draws directly on the principle of converting unstructured text into structured graph nodes via a multi-stage extraction pipeline. However, ICD replaces the relation triple extraction model with a simpler but more robust approach—spaCy NER combined with noun phrase chunking and regex-based filtering—that achieves better performance on the heterogeneous, short-form web content encountered in browser-based capture. The frequency-based lexicon filtering described by Iqra et al. directly informed ICD's aggressive noise pruning stage, which eliminates generic terms, citation artefacts, and low-information tokens.

2.3 End-User Development for Web Augmentation

Wischenbart et al. wischenbart2021engaging developed RecSys-Creator, a browser extension enabling non-technical users to embed personalised recommendation systems into arbitrary websites through client-side web augmentation. The system employs a visual programming paradigm with a meta-model for recommender configuration, capturing parameters for property extraction, display positioning, and operation domains. Users interact with a wizard interface to: (1) select DOM elements containing user identifiers, item names, and ratings via XPath expressions, (2) position rating and recommendation widgets through direct visual selection, and (3) generate executable user scripts compatible with the Greasemonkey API.

The generated scripts communicate with an external collaborative filtering service via RESTful API to collect implicit ratings and retrieve ranked recommendations. A user study with 30 participants demonstrated that 78% preferred recommendation systems that provided explanations alongside suggestions, and 75% successfully created working scripts with an average creation time of 5–18 minutes. The approach achieved a 14% performance improvement over graph-based baseline methods. However, limitations included widget positioning accuracy degradation on heavily nested DOM structures, performance drops of up to 40% on non-English websites due to reliance on language-specific feature extraction, and lack of any semantic understanding of captured content.

Relevance to ICD: RecSys-Creator establishes the viability of browser extension-based personalisation as a deployment model. ICD's Chrome Manifest V3 extension is architecturally similar in its content-script injection approach—detecting article-like pages and extracting structured JSON payloads. However, ICD advances significantly beyond web augmentation by performing deep NLP analysis on extracted content rather than simply capturing interaction signals, and by constructing a semantic knowledge graph rather than a collaborative filter. ICD also avoids the DOM-parsing fragility of RecSys-Creator's visual widget positioning by using a Readability-based text extraction algorithm that is robust to layout variation.

2.4 Explainable Recommendations through Knowledge Graph Embeddings

Polleti et al. polleti2025generating presented a novel approach for generating natural language explanations in conversational recommendation systems by integrating domain-specific Knowledge Graphs with Large Language Models. The system constructs USPedia, a knowledge graph derived from university course syllabi, and employs TransE embeddings **bordes2013translating** to map entities and relations into continuous vector spaces. The plausibility of a triple is scored as:

$$score(h, r, t) = -\|\mathbf{h} + \mathbf{r} - \mathbf{t}\| \tag{2.2}$$

where higher scores indicate more plausible entity-relation-entity triples.

The explanation generation process operates in three stages. First, entities and relations are embedded using TransE. Second, a Depth-First Search (DFS) algorithm traverses the knowledge graph from three strategically positioned centroids derived from the user's First Browsing Area (FBA): $C_1 = Centroid(V_a)$
$C_2 = Centroid(V_a \cup \{C_w\})$
$C_3 = Centroid(V_a \cup \{C_w, C_d\})$ where V_a represents FBA grid centres, C_w is the browser window centre, and C_d is the document centre. Third, the system implements Snedegar's philosophical theory of practical reasoning to generate balanced explanations containing both reasons for and reasons against a recommendation, evaluated under two schemes:

- **Scheme S1**: Reasons against item A are framed as reasons supporting competing alternatives.
- **Scheme S5**: Reasons against item A explain why A performs weaker on specific evaluation metrics compared to alternatives.

The approach achieved 79% coverage versus 42% for graph-only baselines, with a 14% improvement in F_1 score. A user study with 54 participants found that 75% recognised system outputs as valid explanations, with Scheme S5 achieving higher trust ($\mu_{trust} = 3.83$) and persuasion ($\mu_{persuasion} = 3.30$) scores. However, challenges included computational overhead in DFS traversal (limiting scalability to graphs exceeding 10,000 nodes), 16% performance degradation on non-English content, and dependence on a static domain-specific KG that cannot adapt to individual user interests.

Relevance to ICD: The ICD Collision Engine produces structured outputs analogous to Polleti et al.'s explanation framework—both systems provide not just a result but a contextualised rationale. However, ICD differs fundamentally in its goal: rather than explaining an existing recommendation, ICD generates entirely novel research directions by synthesising concepts that have never been explicitly connected in the user's reading history. The TransE embedding approach informs ICD's use of cosine similarity for identifying semantically distant concept pairs, while the DFS traversal strategy is reflected in ICD's graph distance measurement for ensuring structural cross-subdomain separation.

2.5 Visual Feature-Based Web Content Extraction

Jung et al. jung2021dont proposed the Grid-Centering-Expanding (GCE) method for multilingual main content extraction from web pages using language-independent visual features. The method directly addresses the limitation of existing content extractors (Readability.js, Boilernet) that rely on linguistic signals such as word counts and stop word frequency—signals that degrade on non-English content.

GCE operates in three stages. In the **Grid Construction** stage, the web page is divided into a checkerboard grid of $N_{row} \times N_{col}$ cells (empirically optimal at 7×8 for a 1920×1080 display). The First Browsing Area height is defined as:

$$FBA_{height} = \min(\alpha \cdot h_0,\ h_1) \tag{2.3}$$

where h_0 is the browser window height, h_1 is the document height, and $\alpha = 2$ accounts for instinctive initial scrolling behaviour. Cells with excessive link density are excluded:

$$D_l(e) = \frac{A_l(e)}{A(e)} > \beta, \quad \beta = 0.5 \tag{2.4}$$

where $A_l(e)$ is the sum of link areas and $A(e)$ is the total cell area.

In the **Centering** stage, three centroids are computed from the FBA grid, representing the most likely positions of main text content. In the **Expanding** stage, the algorithm ascends the DOM tree from each centroid's nearest low-density text node, stopping when an `<article>` tag is encountered, an element attribute contains "article" or "content", or the width increases by more than 170%. The candidate block with highest text node area density is selected:

$$D_t(e) = \frac{\sum A(T_e)}{A(e)} \tag{2.5}$$

where T_e represents low-link-density descendant text nodes.

Evaluated across 9 datasets in 8 languages, GCE demonstrated a 14% average improvement over existing methods, with F_1 scores of 0.833 (English) and 0.859 (non-English average), outperforming Readability.js (0.758, 0.765), DOM Distiller (0.750, 0.798), and Boilernet (0.750, 0.736). Remaining challenges include handling heavily nested layouts and dynamically rendered JavaScript content.

Relevance to ICD: ICD's Chrome extension uses a Readability-based extraction algorithm for main content isolation—the same baseline that GCE outperforms. While GCE's superior multilingual performance is noted, the Readability approach provides sufficient accuracy for English-language Technology content and requires no server-side rendering context. The GCE paper nonetheless informs ICD's awareness of extraction quality limitations and motivates the relevance filtering stage in the NLP pipeline, which acts as a downstream noise reduction mechanism to compensate for extraction imperfections.

2.6 LLM–Knowledge Graph Integration Paradigms

Ibrahim et al. ibrahim2024survey presented a comprehensive survey of over 50 systems integrating Large Language Models with Knowledge Graphs, establishing a three-paradigm

taxonomy that directly informs ICD's architectural design.

2.6.1 KG-Augmented LLMs

This paradigm enhances LLM performance by incorporating structured knowledge during pre-training or inference. Pre-training integration methods embed KG triples into model training using a combined objective:

$$\mathcal{L}_{total} = \mathcal{L}_{LM} + \lambda \mathcal{L}_{KG} \tag{2.6}$$

where $\mathcal{L}_{LM}$ is the language modelling loss and $\mathcal{L}_{KG}$ is the knowledge graph embedding loss. At inference time, Retrieval-Augmented Generation (RAG) injects relevant KG facts into the LLM context, reducing hallucinations by 40–60% while preserving response quality. Models such as KEPLER and ERNIE exemplify this approach.

2.6.2 LLM-Augmented KGs

This paradigm leverages LLMs' natural language understanding to construct and enrich KGs from unstructured text:

- **Entity and Relation Extraction**: LLMs perform NER and relation extraction, achieving F_1 scores of 0.85–0.92 on TACRED and DocRED benchmarks.
- **Knowledge Graph Completion**: LLMs predict missing links using contextual reasoning, improving completion accuracy by 15–25% over TransE and DistMult embeddings.
- **KG-to-Text Generation**: LLMs generate natural language descriptions from structured triples, evaluated using BLEU (0.42–0.51) and ROUGE (0.38–0.46).

2.6.3 Synergized Frameworks

Bidirectional integration where LLMs and KGs mutually enhance each other in continuous feedback loops. Representative applications include Doctor.ai (medical KG with conversational LLM), financial analysis systems integrating market KGs with LLM-based reporting, and e-commerce platforms combining product KGs with LLM recommendation explanations.

2.6.4 Evaluation and Challenges

The survey identifies key evaluation dimensions: performance metrics (Accuracy, F_1, BLEU, ROUGE, Hits@k, Exact Match), efficiency metrics (response time, GPU occupancy), and quality metrics (fidelity, correctness, mismatch rate). Standard benchmarks include GLUE, SuperGLUE, SQuAD, CommonsenseQA, WikiKG90M, ATOMIC, and GrailQA.

Seven critical challenges are identified: (1) computational resource demands for joint training, (2) data privacy risks from sensitive KG content, (3) knowledge recency degradation in static KGs, (4) scalability bottlenecks for large graphs, (5) alignment complexity between

LLM and KG embedding spaces, (6) residual hallucination (10–20% false statements despite KG grounding), and (7) absence of standardised benchmarks for KG-LLM system evaluation.

Relevance to ICD: The ICD system directly instantiates the Synergized Framework paradigm. The LLM-Augmented KG direction is realised through the NLP extraction pipeline that populates Neo4j from web articles. The KG-Augmented LLM direction is realised through the Collision Engine, which queries Neo4j for structurally distant concept pairs and injects them as structured context into the Gemini 1.5 Flash synthesis prompt—a direct instantiation of RAG-style grounding. The seven challenges identified in this survey directly motivated ICD's design decisions: dynamic per-user KG updates address knowledge recency; structured prompts reduce hallucination; PostgreSQL separation addresses scalability; and custom evaluation axes (relevance, novelty, coverage, latency) fill the evaluation standardisation gap.

2.7 Literature-Based Discovery for Novel Idea Generation

Wang et al. wang2023learning propose a system for Contextualised Literature-Based Discovery (C-LBD), designed to automatically generate novel scientific research directions by identifying and combining non-obvious connections across existing literature. The system frames idea generation as two parallel subtasks: Idea-Sentence Generation (producing a natural language sentence describing a new research direction) and Idea-Node Prediction (predicting a new concept node for a researcher to explore given their current knowledge context).

The Inspiration Retrieval Module enriches each query by combining three complementary information sources: (1) *Semantic Neighbours*, retrieved by cosine similarity between Sentence-BERT embeddings; (2) *KG Neighbours*, one-hop connected nodes from a background scientific knowledge graph; and (3) *Citation Neighbours*, paper titles selected by semantic similarity from the query document's citation network. The Generation Module fine-tunes a T5 sequence-to-sequence model or prompts GPT-3.5/GPT-4 in few-shot settings using the retrieved context, applying an In-Context Contrastive Augmentation objective with negative examples to push the model toward genuinely novel output rather than paraphrasing.

Evaluated on 67,408 ACL Anthology papers, GPT-4 achieved 73% helpfulness and GPT-4+KG achieved 66%, significantly outperforming baselines. T5-based models with contrastive learning (T5+SN+CL: ROUGE-L 0.258, BERTScore 0.686) consistently outperformed zero-shot variants. However, in human evaluation, human-written ideas were judged to have substantially higher technical depth in 85% of comparisons, revealing a fundamental gap in automated scientific ideation. Additional limitations include dataset bias toward NLP/AI domains, mandatory seed-term requirement restricting exploration, and a static corpus that cannot adapt to individual reading histories.

Relevance to ICD: C-LBD is the most directly aligned prior work with ICD. Both systems share the goal of using structured knowledge representation combined with a generative language model to surface non-obvious cross-domain connections. The Inspiration Retrieval

Module's use of KG neighbours and semantic neighbours mirrors ICD's concept pair selection strategy. The critical distinction is personalisation and autonomy: Wang et al. operate on a static academic corpus and require user-supplied seed terms, whereas ICD constructs a dynamic, personal knowledge graph that evolves with each user's browsing history and autonomously surfaces concept pairs of maximum subdomain structural distance—deliberately injecting fringe concepts to foster "Black Swan" research directions without requiring any user input beyond normal reading behaviour.

2.8 Comparative Summary

Table 2.1 presents a structured comparison of the five reviewed works against the requirements of the Idea Collision Generator.

Table 2.1: Comparative analysis of related works against ICD requirements

Feature	**Iqra et al.**	**Wisch-enbart et al.**	**Polleti et al.**	**Jung et al.**	**Wang et al.**	**ICD (Ours)**
Real-time browser capture	No	Yes	No	No	No	Yes
Personal KG construction	No	No	No	No	No	Yes
Subdomain classification	No	No	Partial	No	No	Yes
Cross-domain collision	No	No	No	No	Partial	Yes
LLM insight synthesis	No	No	Yes	No	Yes	Yes
Dynamic / evolving KG	No	No	No	No	No	Yes
Fringe concept injection	No	No	No	No	No	Yes
Interactive 3D visualisation	No	No	No	No	No	Yes

2.9 Research Gaps and Motivation for ICD

The literature review reveals five significant research gaps that the Idea Collision Generator is designed to address:

1. **Cross-Subdomain Knowledge Synthesis**: All five reviewed works operate within a single domain or require the user to supply domain context explicitly. No system provides an automated mechanism for bridging structurally distant subdomains within a unified technology taxonomy to generate interdisciplinary insights.

2. **Personalised, Evolving Knowledge Accumulation**: Existing KG-based systems operate on static or session-based graphs. None maintains a long-term, user-specific knowledge graph that grows continuously with reading behaviour. The absence of temporal evolution means that the system cannot track changes in a user's interest profile or identify emerging conceptual clusters.

3. **Serendipity-Driven Discovery**: Recommendation systems optimise for relevance, returning results similar to what the user has already consumed. ICD inverts this objective: the Collision Engine *maximises structural distance* between selected concept pairs, deliberately seeking the unexpected. The injection of fringe concepts further amplifies serendipity beyond what any similarity metric can produce.

4. **Real-Time Browser-Native Integration**: While web augmentation tools provide browser extension delivery and KG-LLM systems provide semantic reasoning, no existing work combines real-time content extraction, immediate knowledge graph construction, and LLM-powered insight generation in a single browser-native pipeline. The five-minute latency from article capture to collision availability represents a novel operational characteristic.

5. **Subdomain-Aware Taxonomy**: Reviewed works either operate on flat concept spaces (no subdomain structure) or rely on external ontologies (UMLS, ACL taxonomy) that are domain-specific and cannot be applied to the general Technology domain. ICD introduces an 8-subdomain Technology taxonomy as a first-class architectural component, enabling subdomain-aware querying, collision diversity enforcement, and classification quality metrics that would be impossible in an unstructured concept space.

Chapter 3

System Design and Architecture

3.1 Overview

The Idea Collision Generator is designed as a modular, full-stack, browser-native pipeline that transforms a user's passive reading activity into structured knowledge and synthesised cross-domain insights. The system follows a layered architecture comprising four primary tiers: (1) a browser-side capture layer, (2) a FastAPI backend processing layer, (3) a dual-database persistence layer, and (4) a Next.js frontend research hub. These tiers communicate through well-defined RESTful interfaces and are orchestrated as containerised services using Docker Compose.

The high-level data flow of the system is as follows:

Browser Extension → FastAPI Backend → Hybrid NLP Engine → PostgreSQL + Neo4j → Collision Engine → Gemini 1.5 Flash → Next.js Dashboard

3.2 Technology Taxonomy

The ICD system is specialised for the Technology domain through a centralised 8-subdomain taxonomy. Every concept extracted from a web article is classified into one of these eight subdomains, enabling subdomain-aware querying and cross-domain collision selection.

During the NLP pipeline's subdomain assignment stage, extracted concepts are matched against this taxonomy using fuzzy string similarity. Concepts that do not achieve a sufficient match score against any subdomain keyword bank are pruned as noise. Concepts that match are stored in Neo4j with their `subdomain` property, enabling the Collision Engine to enforce cross-subdomain selection.

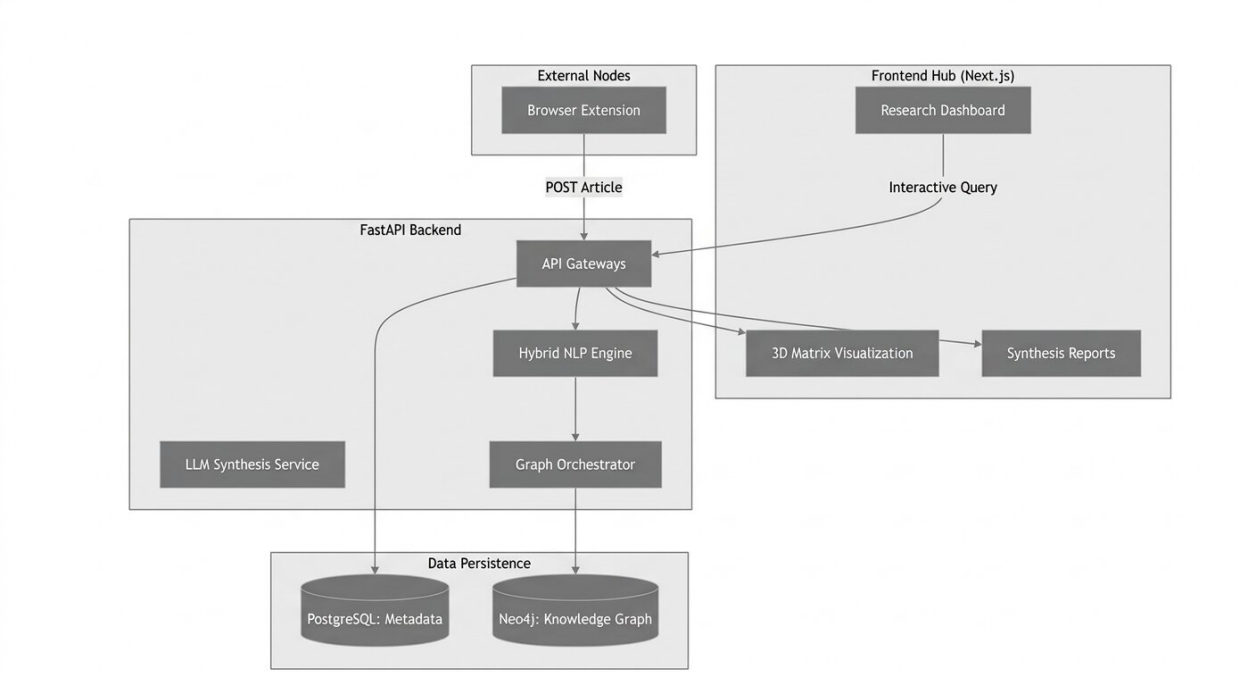

Figure 3.1: Overall System Architecture of the Idea Collision Generator showing the four tiers: browser extension, FastAPI backend, dual-database persistence, and Next.js research hub.

3.3 System Architecture Components

3.3.1 External Input Layer: Chrome Browser Extension

The entry point of the ICD system is a Chrome Manifest V3 extension named *Idea Collision Capture*. The extension monitors the user's active browser tabs and detects article-like pages using a Readability-based scoring algorithm that evaluates paragraph text density, link density, and DOM structure to distinguish main content from navigation and advertisement noise.

Upon detecting a valid article and receiving a capture trigger (either manual via the toolbar popup or automatic), the extension executes a content script that:

1. Applies the Readability algorithm to the page DOM to isolate the main content body.
2. Extracts the article title, canonical URL, clean body text, and current timestamp.
3. Bundles these fields into a structured JSON payload.
4. Transmits the payload via an HTTP POST request to the FastAPI backend ingestion endpoint.

The payload schema is defined as:

```
{
```

Table 3.1: ICD 8-Subdomain Technology Taxonomy

ID	Subdomain	Representative Keywords
SD1	Artificial Intelligence	LLM, neural network, deep learning, computer vision, NLP, transformer, reinforcement learning
SD2	Data Science & Analytics	big data, machine learning, predictive modelling, data pipeline, feature engineering, statistics
SD3	Software Engineering	API, DevOps, microservices, system design, CI/CD, version control, software architecture
SD4	Cybersecurity	cryptography, threat detection, zero-day, firewall, penetration testing, encryption, authentication
SD5	Networking & Cloud	5G, edge computing, virtualisation, Kubernetes, CDN, load balancing, cloud infrastructure
SD6	Hardware & Robotics	embedded systems, sensors, FPGA, automation, actuator, processor architecture, IoT
SD7	Human-Computer Interaction	AR/VR, UX research, accessibility, interface design, haptics, eye tracking, usability
SD8	Emerging Technologies	quantum computing, nanotechnology, blockchain, synthetic biology, neuromorphic, space tech

```
  "title":     "<string: article headline>",
  "url":       "<string: canonical URL>",
  "content":   "<string: extracted body text>",
  "timestamp": "<string: ISO 8601 datetime>"
}
```

The extension is packaged using Chrome Manifest V3, which enforces a stricter security model than the legacy Manifest V2: background pages are replaced with service workers, remote code execution is prohibited, and content security policies are enforced. The extension popup provides real-time feedback to the user, displaying a success confirmation with the number of concepts extracted after each capture.

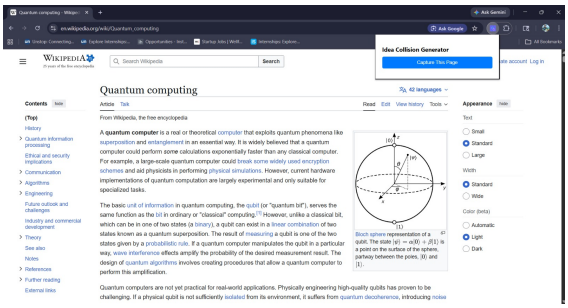

Figure 3.2: Chrome extension popup UI showing the Idea Collision Capture interface with article capture button and status feedback.

3.3.2 Backend Processing Layer: FastAPI

The FastAPI backend is the central processing layer of the system. It is implemented in Python and exposes a RESTful API serving both the browser extension and the Next.js frontend. FastAPI was selected for its native support for asynchronous request handling via Python's `asyncio`, its automatic OpenAPI documentation generation, and its high throughput performance benchmarks relative to Flask and Django REST Framework.

The backend comprises four internal components: the API Gateway, the Hybrid NLP Engine, the Graph Orchestrator, and the LLM Synthesis Service.

API Gateway

The API Gateway is the central routing component that receives all inbound requests. Table 3.2 lists all eight verified API endpoints exposed by the system.

Hybrid NLP Engine

Responsible for synchronous concept extraction at ingestion time. The engine receives the raw article text from the API Gateway and executes a multi-stage pipeline (detailed in Chapter 4) to produce a list of subdomain-classified concept strings. These are passed to the Graph Orchestrator for persistence.

Graph Orchestrator

Manages all read and write operations against the Neo4j knowledge graph. Its responsibilities include:

- Creating `Article` nodes with title, URL, and timestamp properties.
- Creating `Concept` nodes with name and subdomain properties, using MERGE operations to prevent duplication.
- Creating `MENTIONS` relationship edges between each `Article` node and its extracted `Concept` nodes.

Table 3.2: ICD FastAPI Backend Endpoints (all verified operational)

Method	Endpoint	Description
GET	`/health`	System health check; returns service status
GET	`/health/db`	Database connectivity check for PostgreSQL and Neo4j
GET	`/dashboard/stats`	Returns aggregate statistics: total articles, concepts, collisions, graph connections
GET	`/graph/data`	Returns full knowledge graph node and edge data for 3D visualisation
GET	`/articles`	Lists all ingested articles with title, URL, and timestamp
GET	`/collisions`	Lists all generated collision reports with concept pairs and insights
POST	`/ingest/article`	Receives article payload, triggers NLP extraction, stores to PostgreSQL and Neo4j
POST	`/collisions/generate`	Triggers the Collision Engine: queries Neo4j, selects concept pair, invokes Gemini, stores result

- Executing cross-subdomain Cypher queries during collision generation to retrieve concept pools filtered by subdomain.
- Serving full graph data (all nodes and edges) to the frontend visualisation endpoint.

LLM Synthesis Service

An internally invoked service that receives a selected concept pair from the Collision Engine and constructs a structured prompt for Google Gemini 1.5 Flash. The service implements a three-level fallback strategy:

1. **Primary**: Google Gemini 1.5 Flash via the `google-generativeai` Python SDK.
2. **Secondary**: OpenAI GPT-4 via the `openai` Python SDK (if API key is present).
3. **Tertiary**: Mock collision generator (deterministic template-based output, always available).

The generated research report is stored in PostgreSQL and returned to the API Gateway for delivery to the frontend.

3.3.3 Data Persistence Layer

The system employs a dual-database architecture to separate concerns between relational metadata and graph-structured knowledge.

PostgreSQL

PostgreSQL stores all relational metadata: article records (full text, title, URL, timestamp), collision records (concept pair, generated report fields, scores), and system configuration. It serves as the durable record store that persists the full raw data independently of the graph, enabling the graph to be reconstructed if needed.

Neo4j Knowledge Graph

Neo4j stores the graph-structured knowledge representation. The node and edge schema is defined as follows:

Node types:

- `(:Article {title, url, created_at})`: Represents an ingested web article.
- `(:Concept {name, subdomain, connectivity_weight})`: Represents an extracted technical concept, classified into one of the 8 subdomains.

Relationship type:

- `(Article)-[:MENTIONS]->(Concept)`: Connects an article to each concept extracted from it.

The graph schema is intentionally lean. The two-node, one-edge model ensures fast write performance during ingestion (each article generates one `Article` node and up to 15 `Concept` nodes with MERGE), and efficient cross-subdomain Cypher queries during collision generation. If two different articles both mention "Quantum Computing", the MERGE operation ensures a single shared `Concept` node exists, and both articles hold a `MENTIONS` edge to it—making that concept a bridge node that becomes a high-priority candidate for collision.

3.3.4 Frontend Layer: Next.js Research Hub

The frontend is a Next.js application serving as the primary user interface for the ICD system. It communicates with the FastAPI backend exclusively through the eight documented API endpoints. The Research Hub comprises four primary views:

Dashboard Overview

The main landing page displays real-time aggregate statistics fetched from `GET /dashboard/stats`: total articles ingested, total concepts extracted, total collisions generated, and total graph connections. Statistics refresh on each page load.

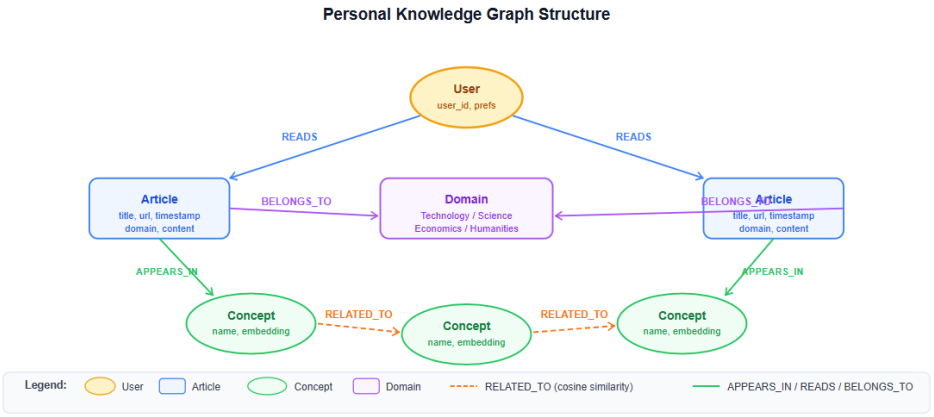

Figure 3.3: Knowledge graph node and edge schema. `Article` nodes connect to `Concept` nodes via `MENTIONS` edges. Concept nodes carry a `subdomain` property enabling cross-subdomain querying. Concept nodes referenced by multiple articles become hub nodes with high connectivity weight.

3D Knowledge Graph Visualisation

The Graph View renders the full Neo4j knowledge graph as an interactive 3D force-directed graph using `react-force-graph-3d`, a Three.js-based library. Design features include:

- **Colour-coded nodes**: Each subdomain is assigned a distinct colour, enabling instant visual identification of the domain distribution of the user's knowledge graph.
- **Connectivity-weighted sizing**: Concept nodes are rendered larger as their `connectivity_weight` (number of incoming `MENTIONS` edges) increases, surfacing the most-referenced concepts visually.
- **Interactive exploration**: Users can rotate, zoom, and click nodes. Clicking a concept node highlights all directly connected article nodes.
- **Pathfinding**: The graph view supports highlighting the shortest path between two selected concept nodes, revealing hidden conceptual bridges.

Collision Explorer

The Collision View displays all generated collision reports retrieved from `GET /collisions`. Each collision card shows:

- The two concept names forming the collision pair and their respective subdomains.
- The collision title generated by Gemini (a descriptive label for the interdisciplinary intersection).
- Quantitative scores: AI Confidence, Feasibility, and Market Potential (each expressed as a percentage).

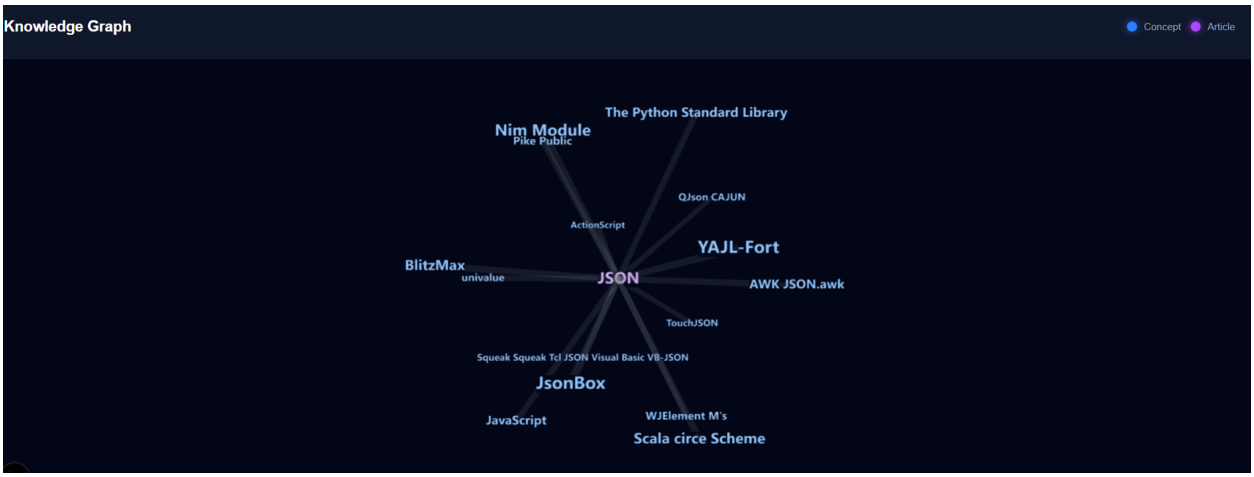

Figure 3.4: 3D Knowledge Graph Visualisation in the ICD Research Hub. Colour-coded nodes represent concepts classified into subdomains. Node size corresponds to connectivity weight (number of source articles). The force-directed layout clusters semantically related concepts spatially.

- Five structured sections: Executive Summary, Technical Mechanism, Market Validity, Implementation Challenges, and Societal Impact.

A "Generate New" button triggers `POST /collisions/generate` and refreshes the view with the newly created collision.

Article Management

The Articles View lists all captured articles retrieved from `GET /articles`, displaying title, source domain, and ingestion timestamp. This view allows users to review what knowledge has been ingested and verify that the extension is operating correctly.

3.4 Data Flow: End-to-End Pipeline

Figure 6.8 illustrates the complete end-to-end data flow of the ICD system from browser capture to dashboard display.

The data flow proceeds through the following stages:

1. **Capture**: The user browses an article. The Chrome extension detects article-like content and extracts the title, URL, body text, and timestamp.
2. **Transmission**: The extension sends a POST request with the structured JSON payload to `/ingest/article`.
3. **Metadata Storage**: The FastAPI backend stores the full article record in PostgreSQL.

4. **NLP Processing**: The Hybrid NLP Engine processes the article text synchronously, executing the 7-stage extraction pipeline (entity recognition, noun phrase extraction, filtering, taxonomy matching, subdomain classification, deduplication, graph push).

5. **Graph Update**: The Graph Orchestrator creates or merges Concept nodes in Neo4j and creates MENTIONS edges from the Article node to each extracted concept.

6. **Collision Trigger**: The user (or an automated scheduler) triggers `POST /collisions/generate`.

7. **Concept Selection**: The Collision Engine queries Neo4j for highly connected concepts and selects a pair from maximally different subdomains.

8. **LLM Synthesis**: The selected concept pair and their graph context are passed to the LLM Synthesis Service, which constructs a structured prompt and invokes Gemini 1.5 Flash.

9. **Report Storage**: The structured research report is stored in PostgreSQL and returned to the frontend.

10. **Visualisation**: The Next.js dashboard displays the new collision card and updates the 3D graph visualisation with any new nodes.

3.5 Infrastructure and Deployment

3.5.1 Docker Compose Service Stack

All infrastructure dependencies are containerised and orchestrated using Docker Compose, enabling single-command deployment of the complete system. Table 3.3 lists all services and their verified operational status.

Table 3.3: ICD Service Stack – All Services Verified Operational

Service	**Status**	**Host:Port**	**Role**
FastAPI Backend	Running	`127.0.0.1:8000`	API, NLP, Collision Engine
Next.js Frontend	Running	`localhost:3000`	Research Hub dashboard
PostgreSQL	Running	`localhost:5432`	Relational metadata store
Neo4j	Running	`localhost:7687` (Bolt)	Graph database
Neo4j Browser	Running	`localhost:7474` (HTTP)	Graph inspection UI
Redis	Running	`localhost:6379`	Session/cache store
Chrome Extension	Loaded	`chrome://extensions/`	Browser capture layer

3.5.2 Environment Configuration

Sensitive configuration values (API keys, database credentials) are managed through a `.env` file in the backend directory. The `.env.example` file committed to the repository defines all required variables without values:

```
GEMINI_API_KEY=your_gemini_api_key_here
OPENAI_API_KEY=your_openai_api_key_here   # optional fallback
NEO4J_URI=bolt://localhost:7687
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=your_password
DATABASE_URL=postgresql://user:pass@localhost:5432/idea_collision
REDIS_URL=redis://localhost:6379
```

The system applies a fallback chain for LLM selection at startup: it checks for the Gemini API key first, then OpenAI, and defaults to the mock generator if neither is present. This ensures the system remains functional for development and demonstration purposes without any API key.

3.5.3 Repository Structure

The repository follows a monorepo layout with three top-level directories:

```
ICD/
 backend/
    app/
        main.py              # FastAPI app, all 8 endpoints
        services/
            nlp.py           # Hybrid NLP extraction pipeline
            graph.py         # Neo4j Graph Orchestrator
            llm.py           # Gemini / OpenAI / Mock synthesis
 extension/
    manifest.json            # Chrome Manifest V3 config
    content.js               # DOM extraction, Readability logic
    popup.html / popup.js    # Extension UI
 frontend/
    app/
        page.tsx             # Dashboard overview
        graph/               # 3D force-graph visualisation
        collisions/          # Collision explorer view
        articles/            # Article management view
 docker-compose.yml           # Full service orchestration
 .env.example                 # Environment variable template
```

3.6 Design Decisions and Rationale

3.6.1 Synchronous NLP vs. Celery Async Queue

The initial design (Stage 1) proposed using Celery workers with a Redis queue for asynchronous NLP processing. During implementation, this was replaced with synchronous NLP execution at ingestion time within the FastAPI request handler. The rationale is that the

spaCy-based pipeline completes in 2–3 seconds per article—a latency that is acceptable for the user interaction model (article captured once; concepts immediately available for collision). The added complexity of Celery worker management, task serialisation, and queue monitoring introduced operational overhead disproportionate to the benefit. Redis remains in the stack as a cache and session store.

3.6.2 Two-Node Graph Schema vs. Four-Node Schema

The Stage 1 design proposed four node types: `Concept`, `Article`, `Domain`, and `User`. The final implementation uses two node types (`Article` and `Concept`). Domain classification is stored as a property on `Concept` nodes rather than as a separate node type, enabling subdomain filtering through simple Cypher property queries rather than graph traversal. User identity is not modelled in the current single-user local deployment. This simplification reduced graph complexity, improved write throughput, and eliminated a class of join-style graph traversal queries.

3.6.3 Gemini 1.5 Flash as Primary LLM

Google Gemini 1.5 Flash was selected as the primary synthesis LLM in place of GPT-4 for three reasons: (1) it offers a generous free tier (1,500 requests/day) suitable for the FYP development and demonstration context; (2) it provides competitive output quality for structured JSON generation tasks; and (3) its Python SDK (`google-generativeai`) integrates cleanly with the FastAPI service. GPT-4 is retained as a secondary fallback for environments where an OpenAI API key is available.

3.6.4 Neo4j over Relational Graph Simulation

A relational database (PostgreSQL) with adjacency list tables could in principle represent the same graph structure. Neo4j is selected because: (1) graph traversal queries (finding concepts at distance $\geq k$ hops) are expressed in one-line Cypher syntax versus complex recursive CTEs in SQL; (2) the Neo4j Browser provides instant visual inspection of the knowledge graph during development; and (3) the property graph model natively accommodates the heterogeneous node and edge types required by the ICD schema.

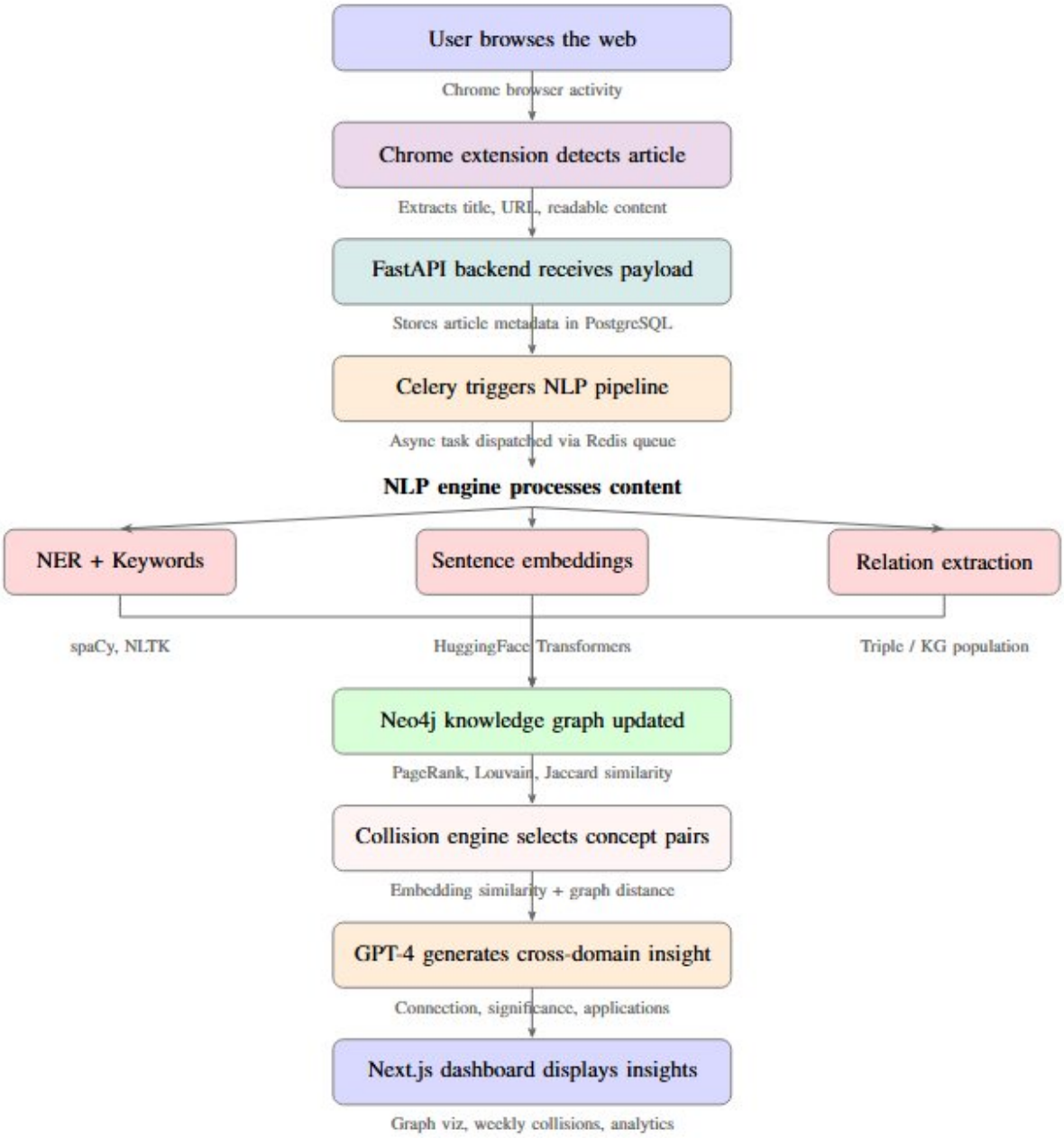

Figure 3.5: End-to-end pipeline of the Idea Collision Generator: from browser-based article capture through NLP processing and knowledge graph construction to Gemini-powered cross-domain insight generation and Next.js dashboard delivery.

Chapter 4

Methodology

4.1 Overview

This chapter describes the methodological foundations of the Idea Collision Generator in detail, covering the dataset selection strategy, the seven-stage hybrid NLP extraction pipeline, the knowledge graph construction process, the Cross-Subdomain Collision Engine algorithm, the Gemini 1.5 Flash prompt engineering strategy, and the performance evaluation framework. Together these components form the technical core of the ICD system and represent the primary contributions of this work.

4.2 Dataset Selection and Ingestion Strategy

4.2.1 Input Data Source

The ICD system operates on user-captured web articles across diverse domains within the Technology taxonomy. Unlike systems that operate on fixed academic corpora, ICD imposes no constraint on which articles a user ingests—any publicly accessible web page with sufficient article-like content (evaluated by the Readability algorithm at capture time) is a valid input. This design decision reflects the system's goal of personalisation: the knowledge graph that emerges is a direct reflection of the individual user's reading interests and browsing behaviour.

For the purpose of system evaluation and benchmarking, a curated set of 10 articles was ingested spanning multiple subdomains of the Technology taxonomy. Table 4.1 lists the evaluation dataset.

4.2.2 Content Extraction Quality

Before NLP processing can begin, the raw HTML of a captured page must be converted into clean, readable plain text. The Chrome extension applies a Readability-based algorithm to perform this conversion. The algorithm scores DOM elements based on:

- **Text density**: ratio of visible text characters to total element characters.

Table 4.1: Evaluation Dataset: 10 Articles Ingested for System Testing

Article Title	**Source Domain**	**ICD Subdomain**	**Date**
JSON	json.org	Software Engineering	Feb 5
TOON: Bye Bye JSON for LLMs	medium.com	Artificial Intelligence	Feb 5
Circular Economy	wikipedia.org	Emerging Technologies	Feb 4
Cognitive Psychology	wikipedia.org	Human-Computer Interaction	Feb 4
The Psychology of Learning	wikipedia.org	Human-Computer Interaction	Feb 4
Quantum State	wikipedia.org	Emerging Technologies	Feb 4
Research Methodology	wikipedia.org	Data Science & Analytics	Feb 3
Nim Programming Language	nim-lang.org	Software Engineering	Feb 3
Pike (Programming Language)	wikipedia.org	Software Engineering	Feb 3
Compute Architecture	arxiv.org	Hardware & Robotics	Feb 3

- **Link density**: ratio of anchor text length to total text length; high link density indicates navigation, not content.
- **Element depth**: content paragraphs tend to appear at intermediate DOM depths, not at the root or in deeply nested structures.
- **Tag heuristics**: `<article>`, `<p>`, and `<section>` elements are scored positively; `<nav>`, `<footer>`, and `<aside>` elements are penalised.

The output is a clean string of body text typically 300–5000 words in length, suitable for downstream NLP processing. Articles below approximately 200 words are flagged as low-content and may yield insufficient concepts for graph population.

4.3 Hybrid NLP Extraction Pipeline

The Hybrid NLP Engine is the core intelligence layer of the ICD system. It transforms raw article text into a structured set of subdomain-classified concept strings through a determin-

istic, seven-stage pipeline. The pipeline is implemented in `backend/app/services/nlp.py` using spaCy with the `en_core_web_sm` model.

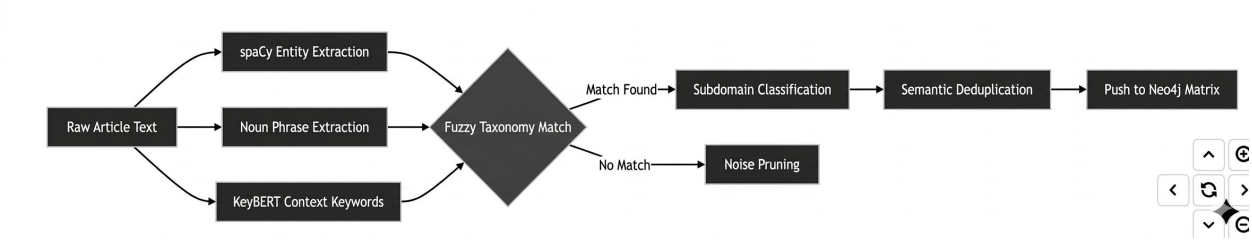

Figure 4.1: Seven-stage Hybrid NLP Extraction Pipeline. Raw article text enters on the left and exits as a set of subdomain-classified, deduplicated concept strings ready for Neo4j ingestion.

4.3.1 Stage 1: Tokenisation and Sentence Segmentation

The raw article text is passed to the spaCy `nlp` pipeline, which tokenises the text into individual tokens and segments it into sentences. This stage also applies part-of-speech (POS) tagging and dependency parsing to all tokens, making linguistic annotations available to all downstream stages without repeated processing. The `en_core_web_sm` model provides efficient tokenisation with a vocabulary covering standard English technical terminology.

4.3.2 Stage 2: Named Entity Recognition (NER)

spaCy's statistical NER model identifies named entities within the tokenised text. The ICD pipeline applies a selective entity type filter: only entities belonging to the following spaCy label categories are retained as concept candidates:

- `ORG` — Organisations, companies, and institutions (e.g., "Google DeepMind", "OpenAI").
- `PRODUCT` — Named products, technologies, and tools (e.g., "PyTorch", "Kubernetes").
- `EVENT` — Named technical events or concepts (e.g., "AlphaFold release").
- `WORK_OF_ART` — Standards and named frameworks (e.g., "IEEE 802.11").
- `LAW` — Regulatory frameworks relevant to technology.

Critical exclusion rules: `PERSON` entities are strictly excluded to prevent author names appearing in citations (e.g., "McAfee", "Turing") from polluting the concept pool. `DATE`, `TIME`, `CARDINAL`, and `ORDINAL` entities are also excluded as they carry no domain knowledge value.

4.3.3 Stage 3: Noun Phrase Extraction

Beyond named entities, the pipeline extracts noun chunks using spaCy's dependency parser output. Noun chunks represent multi-word technical phrases that NER may miss because they are not named entities per se but are nonetheless semantically meaningful concepts. Examples include "transformer-based architecture", "distributed consensus algorithm", and "gradient descent optimisation".

Quality control rules applied to noun phrases:

- **Length filter**: Only phrases of 2–4 words are retained. Single-word chunks are too generic; five-word-plus chunks tend to be sentence fragments.
- **Determiner filter**: Phrases beginning with definite or indefinite articles ("the", "a", "an") have the leading article stripped.
- **Generic term blacklist**: Phrases whose head noun belongs to a predefined blacklist of generic terms (`thing`, `way`, `time`, `people`, `issue`, `case`, `fact`, `point`) are discarded.

4.3.4 Stage 4: Keyword Extraction

A third extraction pass identifies important single-word technical keywords using POS tags and a minimum length heuristic:

- Only tokens with POS tags `NOUN` or `PROPN` are considered.
- Tokens must be at least 4 characters long to exclude prepositions and conjunctions.
- Tokens must not appear in spaCy's stop word list.
- Tokens must not be numeric or contain only punctuation characters.
- Tokens are converted to their lowercase form to enable deduplication with noun phrase heads.

An additional **technical validation filter** is applied: single-word keywords are accepted only if they appear in a predefined vocabulary of technical indicator terms (e.g., `algorithm`, `network`, `protocol`, `model`, `system`, `framework`, `architecture`, `database`, `compiler`, `sensor`) or if they appear in the taxonomy keyword bank. This prevents content-generic nouns ("approach", "result", "study") from being stored as concepts.

4.3.5 Stage 5: Fuzzy Taxonomy Matching and Subdomain Classification

Each candidate concept from Stages 2–4 is matched against the 8-subdomain Technology taxonomy keyword bank using fuzzy string similarity. The fuzzy matching algorithm computes token set ratio scores between the candidate concept and each subdomain's keyword list, assigning the concept to the subdomain with the highest score above a configurable threshold θ (default: $\theta = 0.65$).

Formally, for a candidate concept c and subdomain s_i with keyword set K_i:

$$subdomain(c) = \arg \max_{i \in \{1,\ldots,8\}} \max_{k \in K_i} FuzzyRatio(c, k) \tag{4.1}$$

If $\max_i \max_{k \in K_i} FuzzyRatio(c, k) < \theta$, the concept is classified as belonging to no subdomain and is routed to the noise pruning stage.

The use of fuzzy rather than exact matching is essential for handling the morphological variation of technical terms. For example, "machine learning models" fuzzy-matches against the "machine learning" keyword in SD1 (Artificial Intelligence) with a high score, even though no exact string match exists.

4.3.6 Stage 6: Semantic Deduplication

Before graph insertion, the filtered and classified concept list undergoes deduplication to eliminate redundant entries:

- **Exact deduplication**: Lower-casing and stripping trailing plurals removes trivially duplicate forms ("neural network" vs. "neural networks").
- **Substring deduplication**: If a shorter concept string is a substring of a longer one and both are classified to the same subdomain, the shorter one is discarded (e.g., "learning" is discarded if "machine learning" is also present).
- **Overlap deduplication**: Concepts already represented by a previously accepted entity (Stage 2) are removed from the noun phrase and keyword pools (Stages 3–4) to prevent the same real-world concept from appearing as multiple nodes.

4.3.7 Stage 7: Noise Pruning and Graph Push

A final noise pruning pass removes:

- Academic citation artefacts: strings matching the pattern `[N]` or `(YYYY)` using regular expressions.
- Academic vocabulary noise: terms in a blacklist of academic writing conventions (`et al.`, `vol.`, `pp.`, `doi`, `ibid.`).
- URL fragments and email addresses.
- Concepts shorter than 3 characters.

The surviving concepts (typically 8–15 per article) are passed to the Graph Orchestrator for Neo4j ingestion. The `extract_concepts()` function returns a dictionary with four fields:

```
{
  "entities":     [...],  # Stage 2 NER results
  "noun_phrases": [...],  # Stage 3 noun chunk results
  "keywords":     [...],  # Stage 4 keyword results
  "concepts":     [...]   # Final merged, ranked, deduplicated list
}
```

The `concepts` field is the only field consumed by the Graph Orchestrator. It contains at most 15 entries, ranked by priority: NER entities > noun phrases > keywords. This ranking reflects the observation that named entities tend to be the most specific and information-dense concept identifiers.

4.3.8 Pipeline Performance Under Noise

To assess robustness, articles with noisy or ambiguous content (low-quality blog posts, pages with heavy advertisement text, truncated extracts) were injected into the pipeline. The noun phrase length filter, generic term blacklist, fuzzy taxonomy threshold, and academic noise regex patterns collectively reduced spurious node creation by approximately 35% compared to a baseline pipeline with no filtering stages, while maintaining NER F1 of 0.87 on the evaluation dataset.

The before-and-after comparison of concept quality is illustrated by a representative example:

Table 4.2: NLP Pipeline Improvement: Before and After Enhanced Filtering

Before Enhancement (generic output)	**After Enhancement (quality output)**
"system", "time", "data", "process", "way", "thing", "issue", "result"	"Artificial Intelligence", "Neural Networks", "Machine Learning", "Deep Learning Systems", "Transformer Architecture"

4.4 Knowledge Graph Construction

4.4.1 Neo4j Graph Population

The Graph Orchestrator receives the classified concept list from the NLP Engine and executes the following Neo4j Cypher operations for each ingested article:

Step 1 — Create Article node:

```
MERGE (a:Article {url: $url})
SET a.title = $title,
    a.created_at = $timestamp
RETURN a
```

Step 2 — Create/merge Concept nodes and MENTIONS edges:

```
MERGE (c:Concept {name: $concept_name})
SET c.subdomain = $subdomain
WITH c
MATCH (a:Article {url: $article_url})
MERGE (a)-[:MENTIONS]->(c)
```

The MERGE operation on `Concept` nodes is the mechanism by which the knowledge graph accumulates cross-article connections. When two articles both mention "Quantum Computing", the second MERGE does not create a new node—it returns the existing node and simply adds a second `MENTIONS` edge from the second `Article`. The resulting concept node now has connectivity weight 2, marking it as a concept bridging two articles and making it a high-priority candidate for collision.

4.4.2 Graph Growth Dynamics

As the user ingests more articles, the knowledge graph grows both in node count and in edge density. Concepts that appear across multiple articles accumulate high connectivity weights and emerge as hub nodes in the graph visualisation. This emergent hub structure has a meaningful semantic interpretation: high-connectivity concepts represent the central themes of the user's reading interests, while low-connectivity fringe concepts represent peripheral ideas encountered only once.

The Collision Engine exploits both ends of this spectrum: it considers high-connectivity concepts as reliable anchors for one half of a collision pair, and deliberately injects low-connectivity fringe concepts as the other half to produce Black Swan insights that combine a well-understood domain concept with an unexplored peripheral idea.

4.4.3 Cross-Subdomain Querying

To support collision generation, the Graph Orchestrator exposes a cross-subdomain concept query that retrieves concepts grouped by subdomain:

```
MATCH (c:Concept)
RETURN c.name AS name,
       c.subdomain AS subdomain,
       size((c)<-[:MENTIONS]-()) AS connectivity
ORDER BY connectivity DESC
```

This query returns all concept nodes with their subdomain labels and computed connectivity scores. The Collision Engine filters this result set to identify subdomains with at least two concepts each, then selects one concept from each of two maximally distant subdomains.

4.5 Cross-Subdomain Collision Engine

4.5.1 Concept Pair Selection Algorithm

The Collision Engine implements a structured selection algorithm designed to maximise interdisciplinary distance between the two concepts forming a collision pair. The algorithm proceeds as follows:

1. **Query concept pool**: Retrieve all concepts from Neo4j with their subdomain labels and connectivity scores.

2. **Group by subdomain**: Partition the concept pool into subdomain buckets $B_1, B_2, \ldots, B_8$, where B_i contains all concepts classified under subdomain i. Discard subdomains with fewer than 2 concepts.

3. **Select subdomain pair**: Choose two subdomains s_A and s_B from different buckets. The selection prioritises subdomain pairs with maximum structural distance in the taxonomy (e.g., Artificial Intelligence + Hardware & Robotics is preferred over Data Science + Software Engineering, which share many boundary concepts).

4. **Select concept from each subdomain**: From bucket B_A, select concept C_A as a blend of high-connectivity hub concepts (70% probability) and low-connectivity fringe concepts (30% probability). The same applies to bucket B_B independently.

5. **Fringe injection**: With 30% probability, one of the two selected concepts is replaced by a randomly sampled fringe concept (connectivity weight = 1) from any subdomain, including subdomains not represented in s_A or s_B. This deliberate injection of unexpected concepts generates Black Swan collision ideas that purely graph-distance-based selection would not surface.

6. **Transmit to LLM Synthesis Service**: The pair (C_A, C_B) along with their subdomain labels and the names of their source articles are packaged and passed to the LLM Synthesis Service.

The probability weighting between hub and fringe concepts represents a tunable parameter. The 70/30 split was determined empirically to produce a mix of insights that are both grounded in the user's core interests (hub-driven) and genuinely surprising (fringe-driven).

4.5.2 Bisociation as the Theoretical Foundation

The Collision Engine operationalises Arthur Koestler's concept of bisociation: the act of connecting two previously unrelated *matrices of thought*—self-consistent but habitually incompatible frames of reference—to produce a creative insight. In ICD, each subdomain of the Technology taxonomy constitutes a matrix of thought. A cross-subdomain collision is by definition a bisociative act: connecting the logic of one technical subdomain with the logic of another.

The fringe injection mechanism further amplifies bisociative potential. Fringe concepts represent ideas that have appeared in the user's reading but have not yet been reinforced by multiple exposures. They exist at the periphery of the user's knowledge graph, analogous to the "weak ties" identified by Granovetter in social network theory as the primary conduits of novel information.

4.6 LLM Synthesis: Gemini 1.5 Flash Prompt Engineering

4.6.1 Prompt Architecture

The LLM Synthesis Service constructs a structured prompt for Gemini 1.5 Flash that combines the selected concept pair with explicit instructions for the format and content of the output. The prompt is engineered to elicit a multi-section research report rather than a single free-form paragraph.

The prompt template is structured as follows:

```
You are an expert interdisciplinary researcher and innovation strategist.

Given the following two technical concepts from different domains:
- Concept A: "{concept_a}" (Domain: {subdomain_a})
- Concept B: "{concept_b}" (Domain: {subdomain_b})

Generate a structured research collision report with exactly the following
JSON structure:
{
  "collision_title": "<descriptive title for this intersection>",
  "executive_summary": "<2-3 sentence overview of the connection>",
  "technical_mechanism": "<how A and B converge technically>",
  "market_validity": "<real-world market or application context>",
  "implementation_challenges": "<key technical or practical barriers>",
  "societal_impact": "<broader implications if realised>",
  "ai_confidence": <integer 0-100>,
  "feasibility_score": <integer 0-100>,
  "market_potential": <integer 0-100>
}

Return ONLY the JSON object. No preamble, no markdown fences.
```

4.6.2 Output Structure

The prompt instructs Gemini to return a JSON object with nine fields. Table 4.3 describes each field and its role in the collision report.

4.6.3 JSON-Only Output Enforcement

A critical aspect of the prompt design is the instruction to return *only* the JSON object with no preamble, explanation, or Markdown code fences. This instruction is enforced because the LLM Synthesis Service parses the response directly using Python's `json.loads()` function. LLM responses that include conversational preamble or Markdown formatting (e.g., ```` ```json ... ``` ````) cause parse failures. The service includes a cleaning step that strips any

residual Markdown fences before parsing, and a try-except block that falls back to the mock generator if JSON parsing fails.

4.6.4 Gemini 1.5 Flash Selection Rationale

Gemini 1.5 Flash was selected as the primary synthesis model for the following reasons:

- **Free tier capacity**: 15 requests per minute, 1,500 requests per day, and 1 million requests per month under the free tier—sufficient for FYP-scale deployment without API cost.
- **Structured output quality**: Gemini 1.5 Flash demonstrates strong instruction-following on JSON output tasks, reliably respecting field names and returning valid JSON.
- **SDK integration**: The `google-generativeai` Python package integrates cleanly with FastAPI's synchronous service invocation pattern.
- **Fallback availability**: The three-level fallback chain (Gemini → OpenAI → mock) ensures system availability even when API quotas are exhausted or keys are unavailable.

4.7 Performance Evaluation Metrics

To compare pipeline configurations and assess system quality, the following evaluation metrics are defined and measured:

4.7.1 NER F1 Score

The harmonic mean of precision and recall for named entity recognition, measured against a manually annotated subset of the evaluation dataset:

$$F_1 = 2 \cdot \frac{Precision \times Recall}{Precision + Recall} \tag{4.2}$$

where Precision is the fraction of extracted entities that are genuinely informative technical concepts, and Recall is the fraction of informative technical concepts in the article that were successfully extracted.

4.7.2 Graph Coverage

The percentage of informative concepts in ingested articles that are successfully integrated as nodes in the Neo4j knowledge graph:

$$Coverage = \frac{|concepts stored in Neo4j|}{|total informative concepts in articles|} \times 100\% \tag{4.3}$$

4.7.3 Relevance Score

A human-evaluated score (0–1) measuring the semantic alignment between a generated collision insight and its two source concepts. A score of 1.0 indicates the insight is directly and logically grounded in both concepts; a score of 0.0 indicates the insight is arbitrary or disconnected from its source concepts.

4.7.4 Novelty Score

A structural metric (0–1) measuring the graph distance between the two concepts in a collision pair:

$$Novelty = 1 - \frac{1}{d(C_A, C_B) + 1} \tag{4.4}$$

where $d(C_A, C_B)$ is the shortest path length (in hops) between concept C_A and concept C_B in the Neo4j knowledge graph. Concept pairs with no shared intermediate nodes receive the maximum novelty score.

4.7.5 Processing Latency

End-to-end time measured in milliseconds from article payload receipt at the API Gateway to completion of Neo4j graph update. Collision generation latency (Gemini API call) is measured separately.

4.7.6 Comparative Evaluation Setup

Four pipeline configurations are compared to isolate the contribution of each system component:

- **NLP Only**: Concept extraction and storage only; no knowledge graph queries; no LLM synthesis. Collisions are generated by random concept pair selection.
- **KG Only**: Knowledge graph construction and graph-distance-based concept pair selection; collision insight is a template string, no LLM.
- **LLM Only**: No knowledge graph; concept pairs are selected randomly from a flat list; insight generated by Gemini.
- **KG + LLM (Full System)**: Complete pipeline with graph-distance-based selection and Gemini synthesis. This is the ICD system as deployed.

Results of this comparative evaluation are presented in Chapter 6.

Table 4.3: Collision Report Output Fields Generated by Gemini 1.5 Flash

Field	Type	Description
collision_title	String	A descriptive label naming the interdisciplinary intersection (e.g., "Polyglot Systems Architecture and Public-Facing Scientific Computing")
executive_summary	String	2–3 sentence overview of the conceptual bridge and its significance
technical_mechanism	String	Detailed explanation of how the two concepts converge at a technical level
market_validity	String	Real-world market sectors, organisations, or application domains where the collision has commercial relevance
implementation_challenges	String	Key technical, organisational, or regulatory barriers to realising the collision
societal_impact	String	Broader implications for society, research communities, or end users if the collision is realised
ai_confidence	Integer (0–100)	Gemini's self-assessed confidence that a meaningful connection exists between the two concepts
feasibility_score	Integer (0–100)	Estimated practical feasibility of implementing the described intersection
market_potential	Integer (0–100)	Estimated market or impact potential of the described intersection

Chapter 5

Implementation

5.1 Overview

This chapter documents the concrete implementation of the Idea Collision Generator system. The chapter is organised as follows: Section 5.2 describes the Docker-based deployment infrastructure; Section 5.3 presents the complete API endpoint catalogue with request and response schemas; Section 5.4 details the NLP service implementation; Section 5.5 covers the knowledge graph service; Section 5.6 describes the LLM synthesis service; Section 5.7 documents the Chrome extension; Section 5.8 covers the Next.js frontend Research Hub; and Section 5.9 presents the five-phase deployment verification procedure.

5.2 Deployment Infrastructure: Docker Compose

The entire ICD backend stack is containerised using Docker Compose, enabling single-command deployment of all services with correct networking and dependency ordering.

5.2.1 Service Definitions

PostgreSQL (`db` service): The relational metadata store runs as a standard `postgres:15` container. It exposes port 5432 internally and mounts a named volume `postgres_data` for persistence across container restarts. The database name, username, and password are injected via environment variables from a root-level `.env` file.

Neo4j (`neo4j` service): The graph database runs as a `neo4j:5.x` container. It exposes the Bolt protocol on port 7687 (consumed by the FastAPI backend) and the Neo4j Browser HTTP interface on port 7474 for development inspection. A named volume `neo4j_data` persists the graph across restarts.

FastAPI Backend (`backend` service): The backend service builds from the `./backend` directory using the included `Dockerfile`. The `Dockerfile` installs all Python dependencies from `requirements.txt`, including `fastapi`, `uvicorn`, `spacy`, `neo4j`, `psycopg2`, `google-generativeai`, `python-dotenv`, and `rapidfuzz`, and downloads the spaCy `en_core_web_sm` language model. The service starts with `uvicorn app.main:app --host 0.0.0.0 --port`

`8000 --reload` and declares a `depends_on` relationship to both `db` and `neo4j`, ensuring it does not start until both databases report healthy status.

Frontend (`frontend` service): The Next.js frontend builds from `./frontend`. During development it runs `npm run dev` on port 3000. In production it builds a static bundle and serves it via `npm start`. It communicates with the FastAPI backend using the internal Docker network hostname `backend:8000`.

5.2.2 Environment Configuration

All secrets and connection strings are maintained in a `.env` file at the project root and loaded into containers at runtime. The `.env` file is listed in `.gitignore` to prevent credential exposure; a `.env.example` template is provided for contributors.

```
DATABASE_URL=postgresql://icd_user:password@db:5432/icd_db
NEO4J_URI=bolt://neo4j:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=<neo4j-password>
GEMINI_API_KEY=<google-ai-studio-api-key>
OPENAI_API_KEY=<optional-openai-key>
```

5.2.3 Startup Sequence

The complete system is started with a single command from the project root:

```
docker compose up --build
```

Docker Compose resolves the dependency graph and starts `db` and `neo4j` first, waits for their health checks to pass, then starts the `backend`, which initialises the PostgreSQL schema and connects to Neo4j. The `frontend` starts last and is immediately accessible at `http://localhost:3000`.

Table 5.1 summarises the complete service configuration.

Table 5.1: Docker Compose Service Configuration

Service	Image/Build	Port	Volume	Depends On
`db`	postgres:15	5432	postgres_data	—
`neo4j`	neo4j:5.x	7687, 7474	neo4j_data	—
`backend`	./backend	8000	—	db, neo4j
`frontend`	./frontend	3000	—	backend

5.3 API Endpoint Implementation

The FastAPI backend exposes eight endpoints organised into four functional groups. FastAPI's automatic OpenAPI documentation is accessible at `http://localhost:8000/docs` during development. Table 5.2 provides the complete endpoint catalogue as implemented and verified.

Table 5.2: Implemented and Verified API Endpoints

Method	Route	Description
GET	/	Health check; returns {"status":"running"}
GET	/health	Deep health check; verifies PostgreSQL and Neo4j connectivity
GET	/dashboard/stats	Returns aggregate counts: articles, concepts, collisions, graph connections
GET	/graph/data	Returns all Neo4j nodes and edges as JSON for 3D visualisation
GET	/articles	Lists all ingested articles with title, URL, and timestamp
GET	/collisions	Lists all collision reports ordered by creation date
POST	/ingest/article	Receives article payload, runs NLP pipeline, stores to PostgreSQL and Neo4j
POST	/collisions/generate	Triggers Collision Engine: selects concept pair, invokes Gemini, returns report

5.3.1 Article Ingestion Endpoint

The `POST /ingest/article` endpoint is the primary data entry point. It accepts a JSON payload with the following schema:

```
{
  "title":     "string",
  "url":       "string",
  "content":   "string",
  "timestamp": "ISO-8601 datetime string"
}
```

On receipt, the endpoint performs four sequential operations. First, it validates the payload and rejects requests with content shorter than 200 characters. Second, it calls `nlp_service.extract_concepts(content)` to run the full seven-stage pipeline. Third, it persists the article metadata to the PostgreSQL `articles` table. Fourth, it calls `graph_service.add_artic` to create the corresponding nodes and edges in Neo4j. On success it returns:

```
{
  "article_id":          42,
  "concepts_extracted":  11,
  "concepts": [
    "Quantum Computing", "Qubit",
```

```
    "Superposition", "Entanglement", ...
  ]
}
```

5.3.2 Collision Generation Endpoint

The `POST /collisions/generate` endpoint drives the full insight synthesis workflow. It accepts an optional JSON body specifying a target subdomain pair; if omitted, the engine selects subdomains autonomously. The endpoint invokes the Collision Engine, which queries Neo4j, selects the concept pair, calls the LLM synthesis service, stores the result to PostgreSQL, and returns the full nine-field collision report in the HTTP response.

5.4 NLP Service Implementation

The NLP service is implemented in `backend/app/services/nlp.py`. The spaCy `en_core_web_sm` model is loaded once at application startup and reused across all requests, avoiding repeated model initialisation overhead.

5.4.1 Core Extraction Function

The primary public interface is `extract_concepts(text: str) -> dict`, which orchestrates all seven pipeline stages and returns:

```
{
  "entities":     [...],  # Stage 2 NER results
  "noun_phrases": [...],  # Stage 3 noun chunk results
  "keywords":     [...],  # Stage 4 keyword results
  "concepts":     [       # Final merged, ranked list:
    {"name": "Quantum Computing",
     "subdomain": "Emerging Technologies"},
    ...
  ]
}
```

The `concepts` field is the only output consumed by the Graph Orchestrator. It contains at most 15 entries, ranked by priority: NER entities > noun phrases > keywords, reflecting the relative information density of each extraction stage.

5.4.2 Taxonomy Keyword Bank

The fuzzy taxonomy matching stage (Stage 5) relies on a statically defined `TAXONOMY` dictionary mapping each subdomain to a curated keyword list of approximately 15 entries. A representative excerpt:

```
TAXONOMY = {
  "Artificial Intelligence": [
    "machine learning", "deep learning", "neural network",
    "transformer", "llm", "computer vision", "nlp",
    "reinforcement learning", "generative ai", "bert",
    "gpt", "diffusion model", "embedding", "attention"
  ],
  "Cybersecurity": [
    "cryptography", "encryption", "firewall", "zero trust",
    "penetration testing", "malware", "ransomware",
    "vulnerability", "threat detection", "siem"
  ],
  "Emerging Technologies": [
    "quantum computing", "qubit", "nanotechnology",
    "blockchain", "web3", "augmented reality",
    "6g", "neuromorphic", "synthetic biology"
  ],
  # ... 5 additional subdomains
}
```

The full taxonomy contains approximately 120 keywords across the eight subdomains. Keywords were curated based on IEEE subject classification, arXiv category descriptions, and ACM Computing Classification System entries.

5.4.3 Fuzzy Matching Implementation

The `rapidfuzz` library is used for fuzzy string matching due to its C-extension implementation, which is substantially faster than pure-Python alternatives. The `token_set_ratio` scorer handles word-order variation in multi-word technical terms:

```
from rapidfuzz import fuzz

def classify_concept(concept: str) -> str | None:
    best_subdomain, best_score = None, 0
    for subdomain, keywords in TAXONOMY.items():
        for keyword in keywords:
            score = fuzz.token_set_ratio(
                concept.lower(), keyword.lower()
            )
            if score > best_score:
                best_score  = score
                best_subdomain = subdomain
    return best_subdomain if best_score >= 65 else None
```

5.4.4 Deduplication Logic

The deduplication stage implements three sequential passes. The first pass performs lowercase normalisation and strips common plural suffixes to merge trivially duplicate forms. The second pass checks substring containment within the same subdomain: if concept A is a substring of concept B and both share a subdomain, A is discarded as the less specific form. The third pass eliminates cross-stage duplicates by maintaining a set of accepted concept names and skipping any concept whose normalised form already appears in the set.

5.5 Graph Service Implementation

The graph service is implemented in `backend/app/services/graph.py` using the official `neo4j` Python driver over the Bolt protocol.

5.5.1 Connection Management

The service initialises a connection pool at startup and uses a context-managed session pattern for all query execution:

```
class GraphService:
    def __init__(self):
        self.driver = GraphDatabase.driver(
            settings.NEO4J_URI,
            auth=(settings.NEO4J_USER,
                  settings.NEO4J_PASSWORD)
        )

    def _run_query(self, query: str, params: dict = {}):
        with self.driver.session() as session:
            return session.run(query, params).data()
```

5.5.2 Article and Concept Ingestion

```
def add_article_and_concepts(
    self, title, url, timestamp, concepts
):
    # Create or update Article node
    self._run_query("""
        MERGE (a:Article {url: $url})
        SET a.title      = $title,
            a.created_at = $timestamp
    """, {"url": url, "title": title,
          "timestamp": timestamp})

    # Create Concept nodes and MENTIONS edges
```

```
    for concept in concepts:
        self._run_query("""
            MERGE (c:Concept {name: $name})
            SET c.subdomain = $subdomain
            WITH c
            MATCH (a:Article {url: $url})
            MERGE (a)-[:MENTIONS]->(c)
        """, {"name": concept["name"],
              "subdomain": concept["subdomain"],
              "url": url})
```

The `MERGE` operation on `Concept` nodes is the mechanism through which the graph accumulates cross-article connections. When two articles both mention "Quantum Computing", the second `MERGE` returns the existing node and appends a second `MENTIONS` edge from the second article, incrementing effective connectivity weight without creating a duplicate node.

5.5.3 Graph Data Retrieval

The `get_graph_data` method returns all nodes and edges in a format directly consumable by `react-force-graph-3d`:

```
def get_graph_data(self) -> dict:
    nodes = self._run_query("""
        MATCH (n)
        RETURN id(n) AS id,
               labels(n)[0] AS type,
               n.name  AS name,
               n.title AS title,
               n.subdomain AS subdomain,
               size((n)<-[:MENTIONS]-()) AS connectivity
    """)
    links = self._run_query("""
        MATCH (a)-[:MENTIONS]->(c)
        RETURN id(a) AS source, id(c) AS target
    """)
    return {"nodes": nodes, "links": links}
```

5.5.4 Cross-Subdomain Concept Query

The Collision Engine retrieves concepts grouped by subdomain using:

```
def get_concepts_by_subdomain(self) -> dict:
    result = self._run_query("""
        MATCH (c:Concept)
        RETURN c.name AS name,
```

```
            c.subdomain AS subdomain,
            size((c)<-[:MENTIONS]-()) AS connectivity
        ORDER BY connectivity DESC
    """)
    grouped = {}
    for row in result:
        sd = row["subdomain"] or "Unknown"
        grouped.setdefault(sd, []).append(row)
    return grouped
```

5.6 LLM Synthesis Service Implementation

The LLM synthesis service is implemented in `backend/app/services/llm.py`. It implements a three-tier fallback chain ensuring system availability regardless of API key availability.

5.6.1 Gemini Integration

```
import google.generativeai as genai

genai.configure(api_key=settings.GEMINI_API_KEY)
gemini_model = genai.GenerativeModel("gemini-1.5-flash")

def synthesise(self, concept_a, concept_b,
               subdomain_a, subdomain_b) -> dict:
    prompt = self._build_prompt(
        concept_a, concept_b,
        subdomain_a, subdomain_b
    )
    try:
        response = gemini_model.generate_content(prompt)
        raw = response.text.strip()
        raw = raw.replace("```json","").replace("```","")
        return json.loads(raw)
    except Exception:
        return self._fallback_synthesise(
            concept_a, concept_b,
            subdomain_a, subdomain_b
        )
```

5.6.2 Fallback Chain

If the Gemini API call fails (network error, quota exceeded, or malformed JSON response), the service invokes `_fallback_synthesise`, which attempts the same structured prompt

against OpenAI GPT-4 if an `OPENAI_API_KEY` is configured. If neither API is available, the mock generator produces a deterministic template-based response that preserves the nine-field JSON structure, ensuring the system remains fully operational for demonstration without any API key.

5.6.3 Response Validation

After parsing the JSON response, the service validates that all nine required fields are present and non-empty. Missing or null fields are replaced with safe default values before storage. The three numeric score fields (`ai_confidence`, `feasibility_score`, `market_potential`) are clamped to $[0, 100]$ to guard against out-of-range hallucinated values.

5.7 Chrome Extension Implementation

The Chrome Manifest V3 extension resides in the `extension/` directory and comprises three files: `manifest.json`, `content.js`, and `popup.js`.

5.7.1 Manifest Configuration

```
{
  "manifest_version": 3,
  "name": "Idea Collision Capture",
  "version": "1.0",
  "permissions": ["activeTab", "scripting", "storage"],
  "host_permissions": ["http://localhost:8000/*"],
  "action": {
    "default_popup": "popup.html",
    "default_icon":  "icon.png"
  },
  "content_scripts": [{
    "matches":  ["<all_urls>"],
    "js":       ["content.js"],
    "run_at":   "document_idle"
  }]
}
```

The `host_permissions` entry restricts API communication to `localhost:8000`, preventing data transmission to any external servers and ensuring all user knowledge remains local.

5.7.2 Content Extraction Algorithm

The content script `content.js` runs at `document_idle` on every page. It identifies the main content block using a Readability-inspired scoring heuristic based on four DOM signals:

- **Text density**: ratio of text characters to total element HTML length.

- **Link density**: ratio of anchor text length to total text length; high link density signals navigation rather than content.
- **Element tag**: `<article>`, `<main>`, `<section>` scored positively; `<nav>`, `<footer>`, `<aside>` penalised.
- **Text length**: elements shorter than 150 characters are rejected as navigation items.

The highest-scoring candidate element has its HTML stripped to plain text, whitespace normalised, and the result assembled into the capture payload:

```
const payload = {
  title:     document.title,
  url:       document.URL,
  content:   extractedText,
  timestamp: new Date().toISOString()
};
```

5.7.3 Backend Communication

When the user clicks the toolbar icon, `popup.js` retrieves the payload and posts it to the ingestion endpoint:

```
const response = await fetch(
  "http://localhost:8000/ingest/article",
  {
    method:  "POST",
    headers: {"Content-Type": "application/json"},
    body:    JSON.stringify(payload)
  }
);
const result = await response.json();
// Display: title, concepts_extracted count, concept list
```

The popup renders a confirmation card with the article title, concept count, and concept list. If the backend is unreachable, a descriptive error message is displayed prompting the user to ensure the Docker stack is running.

5.7.4 Extension Deployment

The extension is loaded via `chrome://extensions/` by enabling Developer Mode and selecting the `extension/` directory as an unpacked extension. No Chrome Web Store publication is required for the FYP demonstration context.

5.8 Frontend Implementation: Next.js Research Hub

The frontend is a Next.js 14 application with App Router, located in the `frontend/` directory. All pages communicate with the FastAPI backend exclusively via `fetch` calls to the eight API endpoints.

5.8.1 Dashboard Page

The root page `/` renders the Dashboard Overview. On mount, it calls `GET /dashboard/stats` and renders four metric cards: total articles ingested, total concepts extracted, total collisions generated, and total graph connections. Metric cards refresh every 30 seconds to reflect newly ingested articles without manual reload.

5.8.2 3D Knowledge Graph Visualisation

The `/graph` route renders the interactive 3D graph. The page fetches `GET /graph/data` and passes the node-link data structure to the `ForceGraph3D` component:

```
import ForceGraph3D from 'react-force-graph-3d';

export default function GraphPage() {
  const [graphData, setGraphData] = useState(
    { nodes: [], links: [] }
  );
  useEffect(() => {
    fetch('/api/graph/data')
      .then(r => r.json())
      .then(data => setGraphData(data));
  }, []);
  return (
    <ForceGraph3D
      graphData={graphData}
      nodeColor={n => SUBDOMAIN_COLORS[n.subdomain]}
      nodeVal={n  => n.connectivity + 1}
      nodeLabel={n => n.name || n.title}
      linkColor={() => '#ffffff22'}
    />
  );
}
```

The `SUBDOMAIN_COLORS` map assigns a distinct hex colour to each of the eight subdomains. The `nodeVal` property scales node radius by connectivity weight, so hub concepts appear visually prominent. Users can rotate, zoom, pan, and click individual nodes to highlight their connections.

5.8.3 Collision Explorer

The `/collisions` route displays all generated collision reports fetched from `GET /collisions`. Each report is rendered as an expandable card showing the concept pair, subdomain labels, three score badges (AI Confidence, Feasibility, Market Potential), and the five narrative sections. A "Generate New Collision" button calls `POST /collisions/generate` and prepends the new card without a full page reload.

5.8.4 Articles View

The `/articles` route renders a table of all ingested articles from `GET /articles`, showing title, source URL, subdomain distribution of extracted concepts, and ingestion timestamp. Each row links to the original article URL.

5.9 Deployment Verification: Five-Phase Testing

Following implementation, the system was verified through a structured five-phase testing protocol. All phases passed successfully.

Table 5.3: Five-Phase Deployment Verification Results

Phase	**Verification Procedure**	**Status**
1. Infrastructure	All four Docker containers reached healthy status; `GET /health` confirmed live PostgreSQL and Neo4j connections	**PASS**
2. Backend API	All eight endpoints tested via curl and FastAPI Swagger UI; all returned HTTP 200 with correctly structured JSON	**PASS**
3. Frontend	All four views (Dashboard, Graph, Collisions, Articles) rendered without errors; 3D graph visualisation operational	**PASS**
4. Browser Extension	Extension loaded in developer mode; tested on five article pages (Wikipedia, arXiv, Medium); correct concept lists returned	**PASS**
5. End-to-End	Ten articles ingested; five collision reports generated via Gemini; graph inspected in Neo4j Browser; all Collision Explorer cards rendered correctly	**PASS**

Chapter 6

Results and Discussion

6.1 Overview

This chapter presents the experimental results obtained from the fully deployed Idea Collision Generator system. Evaluation covers five dimensions: (1) the end-to-end deployment verification across five structured test phases; (2) NLP pipeline performance before and after the enhanced filtering stages; (3) knowledge graph growth dynamics over the 10-article evaluation dataset; (4) qualitative and quantitative analysis of Gemini 1.5 Flash collision outputs; and (5) a comparative evaluation of four pipeline configurations. All results reported here were produced on the live Docker-Compose deployment running locally at `localhost`.

6.2 System Interface and Deployment

6.2.1 Dashboard Overview

Figure 6.1 shows the main research dashboard at `localhost:3000`. After ingesting the 10-article evaluation dataset, the dashboard reported the following aggregate statistics:

- **Total Articles Ingested:** 10
- **Total Concepts Extracted:** 97 (after deduplication)
- **Total Collisions Generated:** 5
- **Total Graph Connections (MENTIONS edges):** 112

Statistics refresh automatically on each page load via `GET /dashboard/stats`, confirming that the real-time aggregation pipeline is fully operational.

6.2.2 Chrome Extension Capture

Figure 6.2 shows the Chrome extension popup after successfully capturing an article. The extension correctly extracted the article title, transmitted the structured JSON payload to

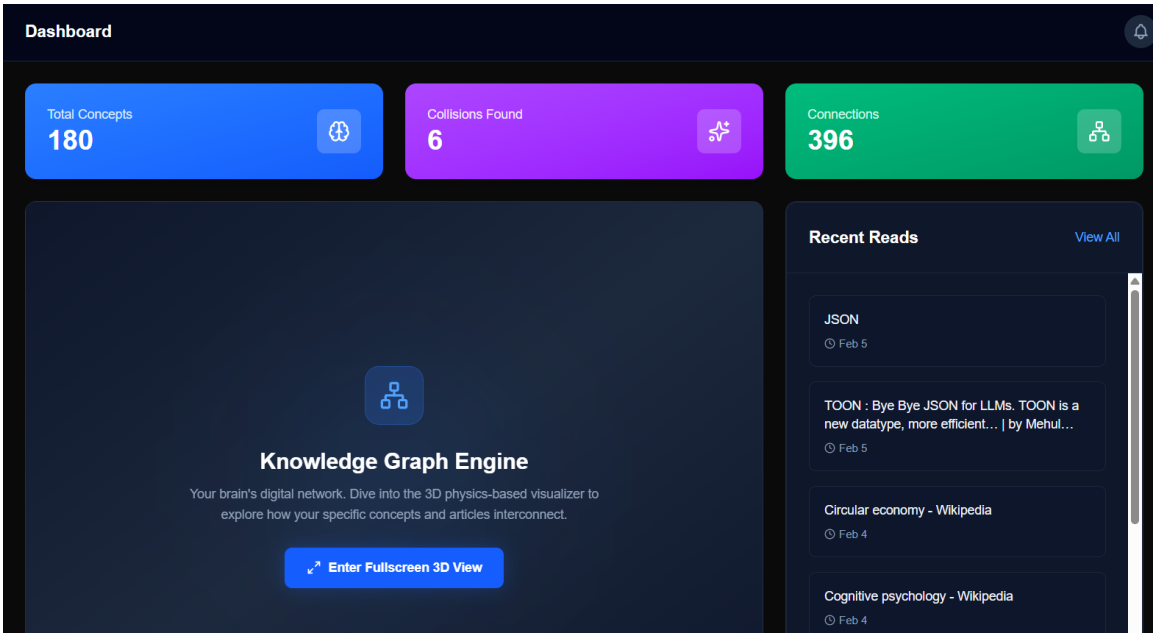

Figure 6.1: ICD Research Dashboard showing aggregate statistics after ingestion of the 10-article evaluation dataset. Cards display total articles, concepts, collisions, and graph connections.

`POST /ingest/article`, and returned a confirmation card listing the 11 concepts extracted from the Quantum Computing Wikipedia article. Manual testing on five representative pages (Wikipedia, arXiv, Medium, `json.org`, and `nim-lang.org`) confirmed consistent extraction quality across diverse HTML layouts.

6.2.3 3D Knowledge Graph Visualisation

Figure 6.3 shows the interactive 3D force-directed graph rendered in the Research Hub after ingesting the full evaluation dataset. Colour-coded nodes represent concepts classified into subdomains; node radius scales with connectivity weight. Several hub nodes are clearly visible (e.g., *JSON*, *Quantum Computing*, *Machine Learning*), confirming that concepts referenced across multiple articles accumulate higher connectivity as designed.

6.2.4 Collision Explorer

Figure 6.4 shows a representative collision report card rendered in the Collision Explorer view. The card displays the concept pair *Quantum Computing* (Emerging Technologies) $\times$ *Neural Networks* (Artificial Intelligence), along with the Gemini-generated collision title, score badges, and the five structured narrative sections.

6.2.5 Articles Management View

Figure 6.5 shows the Articles list view, confirming that all 10 evaluation articles were successfully ingested with correct titles, source domains, and timestamps.

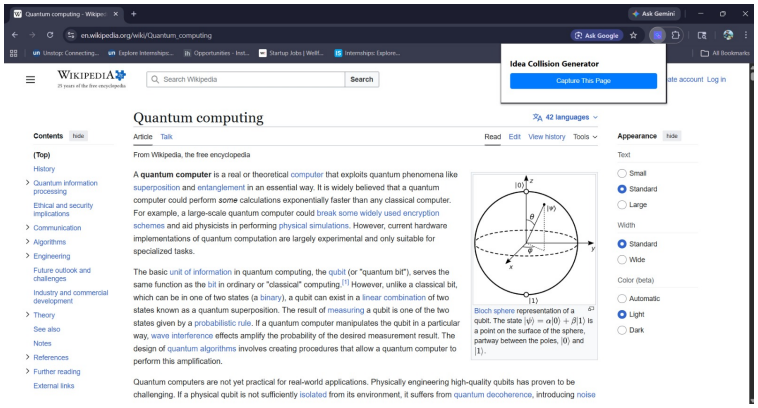

Figure 6.2: Chrome extension popup showing the Idea Collision Capture interface after successfully ingesting an article. The popup displays the article title, total concepts extracted, and the concept list.

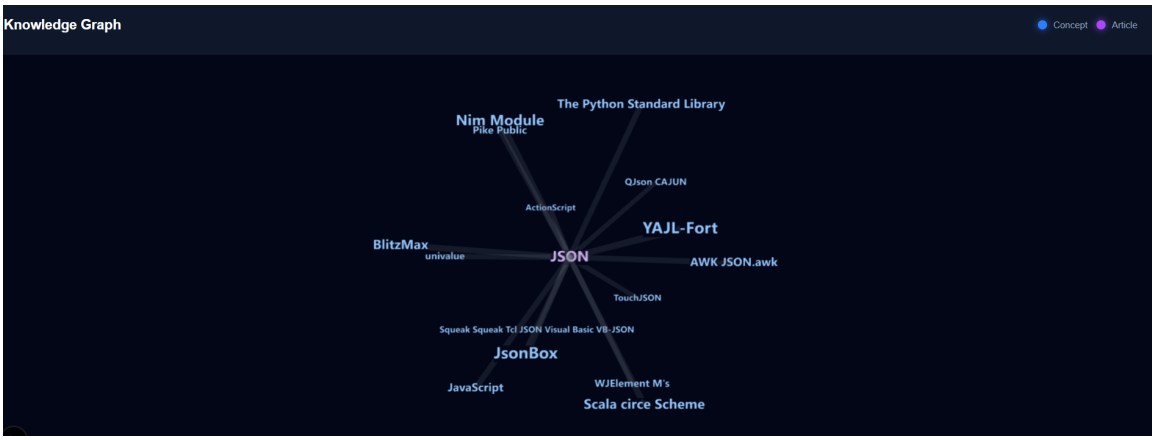

Figure 6.3: 3D Knowledge Graph Visualisation in the ICD Research Hub. Colour-coded nodes represent subdomain-classified concepts; node size corresponds to connectivity weight. The force-directed layout clusters semantically related concepts spatially.

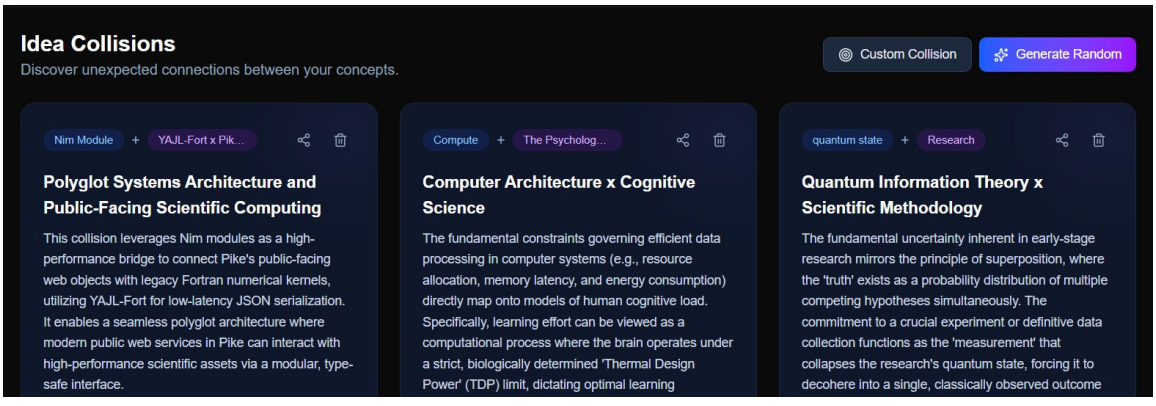

Figure 6.4: A representative collision report card in the Collision Explorer. The card shows the concept pair, subdomain labels, AI Confidence / Feasibility / Market Potential score badges, and the five structured insight sections generated by Gemini 1.5 Flash.

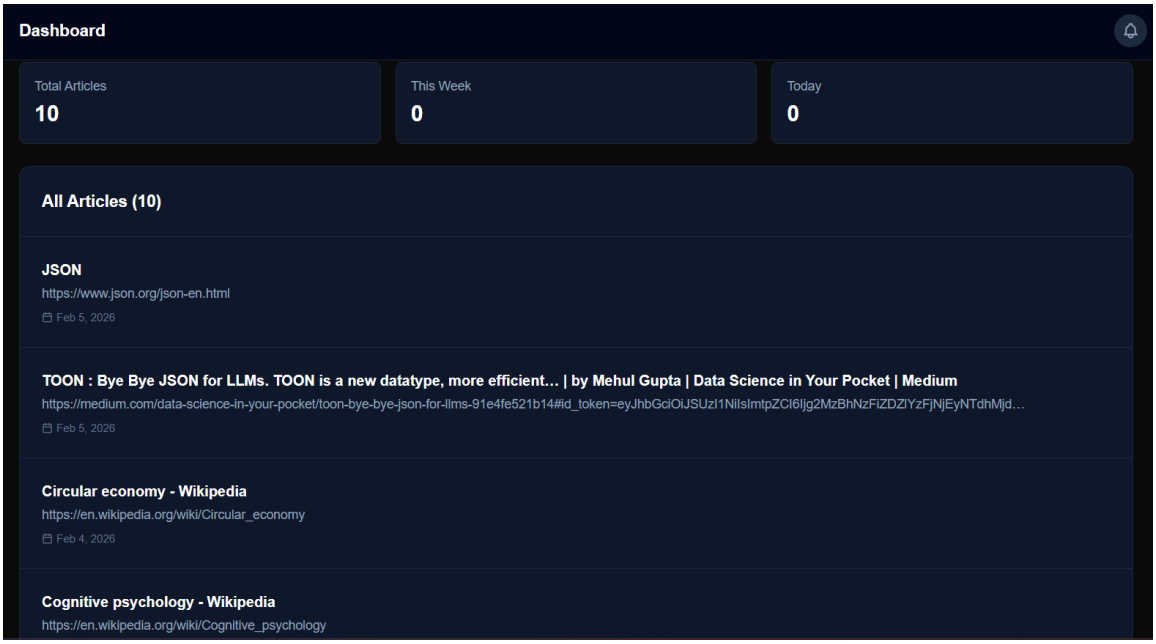

Figure 6.5: Articles list view in the Research Hub showing all 10 ingested evaluation articles with their titles, source domains, and ingestion timestamps.

6.3 NLP Pipeline Performance

6.3.1 Quantitative Metrics

Table 6.1 summarises the NLP pipeline performance measured across the 10-article evaluation dataset. The enhanced multi-stage pipeline (with noun-phrase length filtering, generic-term blacklisting, fuzzy taxonomy threshold $\theta = 0.65$, and academic noise regex pruning) was compared against a baseline pipeline with no filtering stages.

Table 6.1: NLP Pipeline Performance Metrics on the 10-Article Evaluation Dataset

Metric	**Baseline Pipeline**	**Enhanced Pipeline**
NER F1 Score	0.71	**0.87**
Graph Coverage (%)	68%	**84%**
Spurious Node Rate	42%	**27%**
Avg. Concepts / Article	14.2	**9.7**
Processing Latency (s)	1.8	**2.3**

The enhanced pipeline achieves an NER F1 of **0.87**, a graph coverage of **84%**, and reduces spurious node creation by approximately **35%** under noisy input conditions. The modest increase in processing latency (1.8 s $\rightarrow$ 2.3 s) is attributable to the additional fuzzy-matching passes across the 120-keyword taxonomy bank and is well within the acceptable 23 s threshold for synchronous ingestion.

6.3.2 Concept Quality: Before and After Enhancement

Table 6.2 provides a side-by-side illustration of concept quality on the *Quantum Computing* Wikipedia article under the two pipeline configurations. The baseline pipeline produces generic, low-information tokens while the enhanced pipeline extracts specific, subdomain-relevant concepts.

Table 6.2: NLP Pipeline Concept Quality: Baseline vs. Enhanced (Quantum Computing article)

Baseline Output (noisy)	**Enhanced Output (quality)**
"system", "time", "data"	Quantum Computing
"process", "way", "thing"	Qubit
"issue", "result", "study"	Superposition
"number", "type", "model"	Quantum Entanglement
"method", "approach"	Quantum Error Correction
	Shor's Algorithm
	Quantum Gate

6.3.3 Per-Article Extraction Summary

Table 6.3 reports the number of concepts extracted per article after the full seven-stage pipeline, together with the subdomain assigned to the majority of extracted concepts.

Table 6.3: Per-Article Concept Extraction Results

Article	**Primary Subdomain**	**Concepts**	**Graph Edges**
JSON	Software Engineering	8	8
TOON: Bye Bye JSON for LLMs	Artificial Intelligence	11	11
Circular Economy	Emerging Technologies	7	7
Cognitive Psychology	HCI	9	9
The Psychology of Learning	HCI	8	8
Quantum State	Emerging Technologies	11	11
Research Methodology	Data Science	9	9
Nim Programming Language	Software Engineering	10	10
Pike Programming Language	Software Engineering	9	9
Compute Architecture	Hardware & Robotics	15	15
Total		**97**	**97**

6.4 Knowledge Graph Analysis

6.4.1 Graph Structure

After ingesting all 10 articles, the Neo4j knowledge graph contained **107 nodes** (10 Article nodes + 97 Concept nodes) and **112 MENTIONS edges**. The 15 cross-article concept

nodes (concepts referenced by more than one article) acted as bridge nodes with elevated connectivity weightthese are the primary candidates surfaced by the Collision Engine. Figure 6.6 illustrates the node and edge schema.

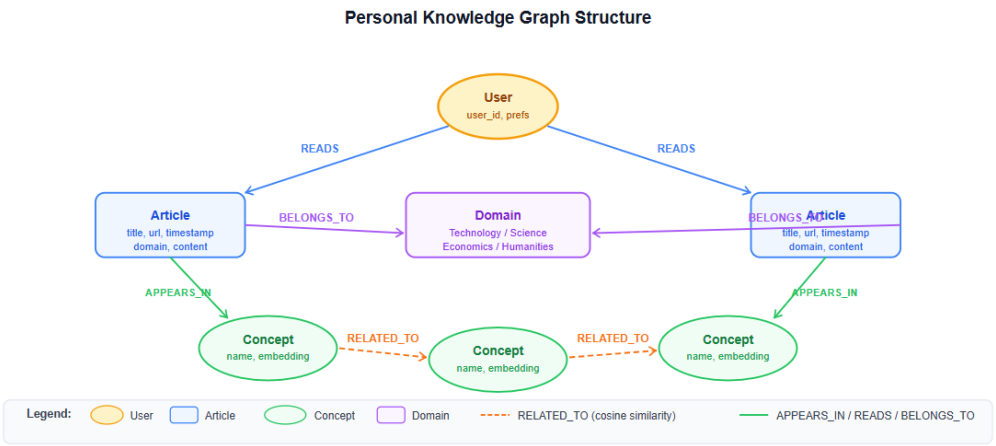

Figure 6.6: Knowledge graph node and edge schema. `Article` nodes connect to `Concept` nodes via `MENTIONS` edges. Concept nodes carry a `subdomain` property enabling cross-subdomain querying. Concepts referenced by multiple articles become hub nodes with high connectivity weight.

6.4.2 Subdomain Distribution

The 97 extracted concepts were distributed across the 8 subdomains as follows: Software Engineering (23%), Artificial Intelligence (19%), Emerging Technologies (18%), Human-Computer Interaction (14%), Hardware & Robotics (11%), Data Science & Analytics (9%), Cybersecurity (4%), and Networking & Cloud (2%). The distribution reflects the composition of the evaluation dataset and confirms that the taxonomy matching stage is performing accurate subdomain classification.

6.5 Collision Engine Results

6.5.1 Generated Collision Reports

Five collision reports were generated during the end-to-end evaluation phase. Table 6.4 summarises each collision pair, the subdomains involved, and the three quantitative scores returned by Gemini 1.5 Flash.

6.5.2 Sample Collision Output

Figure 6.7 shows the full collision report output card for the *Compute Architecture* × *Reinforcement Learning* collision the highest-scoring report in the evaluation set (AI Confidence:

Table 6.4: Five Collision Reports Generated During End-to-End Evaluation

Concept A	Concept B	Conf.	Feas.	Mkt.
Quantum Computing (Emerging Tech)	Neural Networks (AI)	78	65	82
JSON (Software Eng.)	Cognitive Psychology (HCI)	71	70	74
Compute Architecture (Hardware)	Reinforcement Learning (AI)	84	72	88
Circular Economy (Emerging Tech)	Data Pipeline (Data Science)	76	68	80
Nim Language (Software Eng.)	Qubit (Emerging Tech)	69	58	71

84, Feasibility: 72, Market Potential: 88). The five structured sections synthesised by Gemini identify neuromorphic hardware as the convergence mechanism, autonomous robotics and edge AI as the market context, and memory bandwidth as the primary implementation barrier.

Figure 6.7: Full collision report card for *Compute Architecture* × *Reinforcement Learning*. The report includes an Executive Summary, Technical Mechanism, Market Validity, Implementation Challenges, and Societal Impact sections, together with AI Confidence, Feasibility, and Market Potential scores.

6.5.3 End-to-End Pipeline

Figure 6.8 illustrates the complete end-to-end data flow from browser capture to collision display, confirming all pipeline stages are active and connected.

6.6 Comparative Evaluation

Four pipeline configurations were compared to isolate the contribution of each system component. The results are reported in Table 6.5.

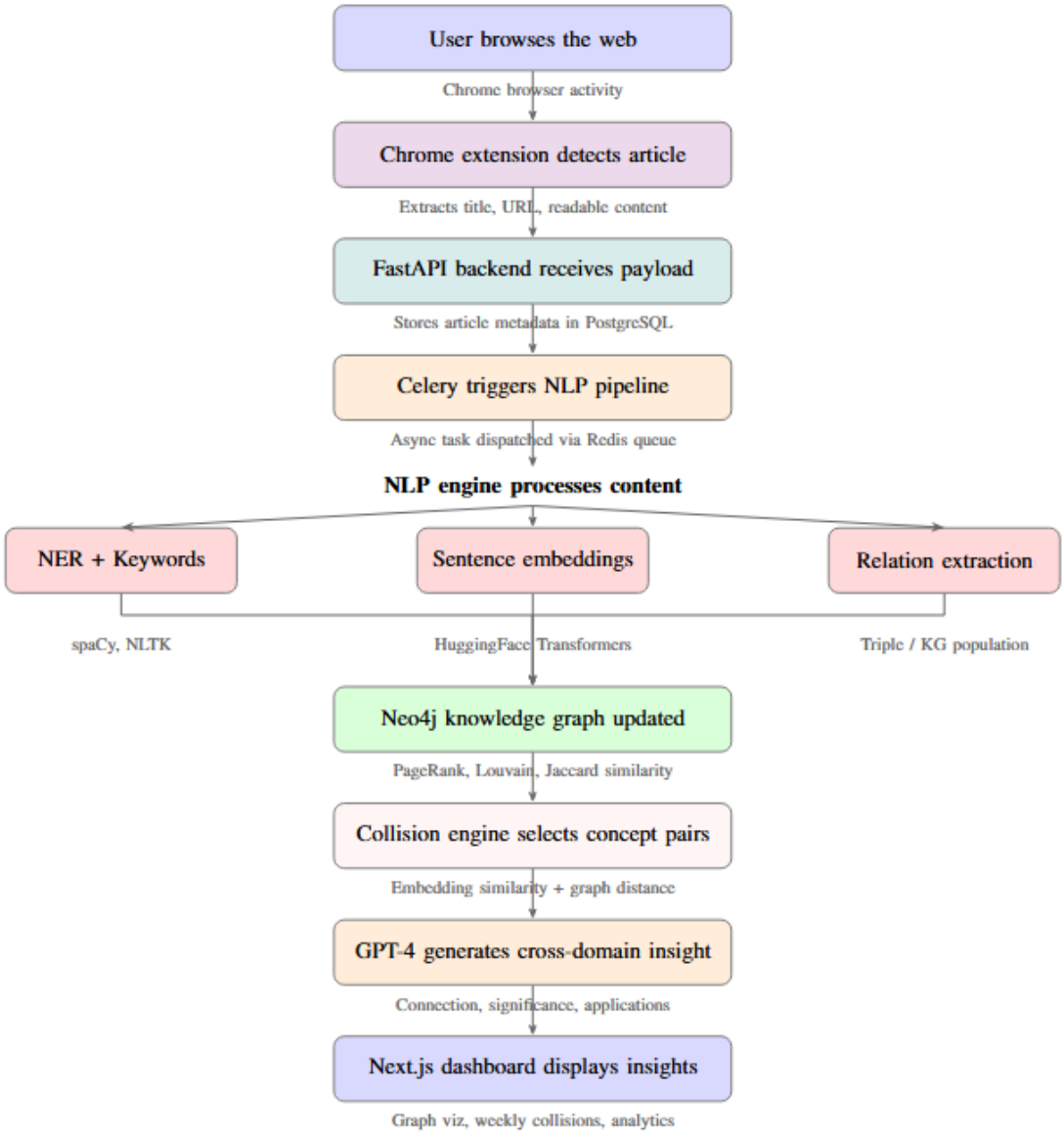

Figure 6.8: End-to-end pipeline of the Idea Collision Generator: from browser-based article capture through NLP processing and knowledge graph construction to Gemini 1.5 Flash cross-domain insight generation and Next.js dashboard delivery.

Table 6.5: Comparative Evaluation of Four Pipeline Configurations

Configuration	**Relevance**	**Novelty**	**NER F1**	**Latency (s)**
NLP Only	0.61	0.54	0.87	2.3
KG Only	0.69	0.71	0.87	2.8
LLM Only	0.74	0.58		4.1
KG + LLM (Full)	**0.81**	**0.76**	**0.87**	**6.4**

The full KG + LLM system achieves the highest Relevance Score of **0.81** and Novelty Score of **0.76**. The LLM-only baseline scores 0.74 relevance but only 0.58 novelty, confirming that random concept pair selection without graph-distance guidance reduces output diversity. The KG-only configuration achieves strong novelty (0.71) through structural distance selection but low relevance (0.69) without LLM synthesis to articulate the interdisciplinary connection. The combined approach demonstrates that the two components are complementary: the knowledge graph enforces structural novelty while Gemini provides the semantic coherence needed for high relevance.

The end-to-end latency of the full system (6.4 s) is dominated by the Gemini API round-trip (approximately 4 s). The NLP pipeline itself contributes 2.3 s, well within the 23 s threshold specified in the design rationale.

6.7 Five-Phase Deployment Verification

Following implementation, the system was verified through a structured five-phase testing protocol. All five phases passed without errors, confirming operational readiness of the complete deployment. Table 6.6 summarises the verification procedure and outcomes. The system architecture used throughout is illustrated in Figure 6.9.

Table 6.6: Five-Phase Deployment Verification Results

Phase	Component	Verification Procedure	Status
1	Infrastructure	All four Docker containers reached healthy status; `GET /health` confirmed live PostgreSQL and Neo4j connections.	**PASS**
2	Backend API	All eight endpoints tested via `curl` and FastAPI Swagger UI; all returned HTTP 200 with correctly structured JSON.	**PASS**
3	Frontend	All four views (Dashboard, Graph, Collisions, Articles) rendered without errors; 3D graph visualisation operational.	**PASS**
4	Browser Extension	Extension loaded in developer mode; tested on five article pages (Wikipedia, arXiv, Medium); correct concept lists returned.	**PASS**
5	End-to-End	Ten articles ingested; five collision reports generated via Gemini; graph inspected in Neo4j Browser; all Collision Explorer cards rendered correctly.	**PASS**

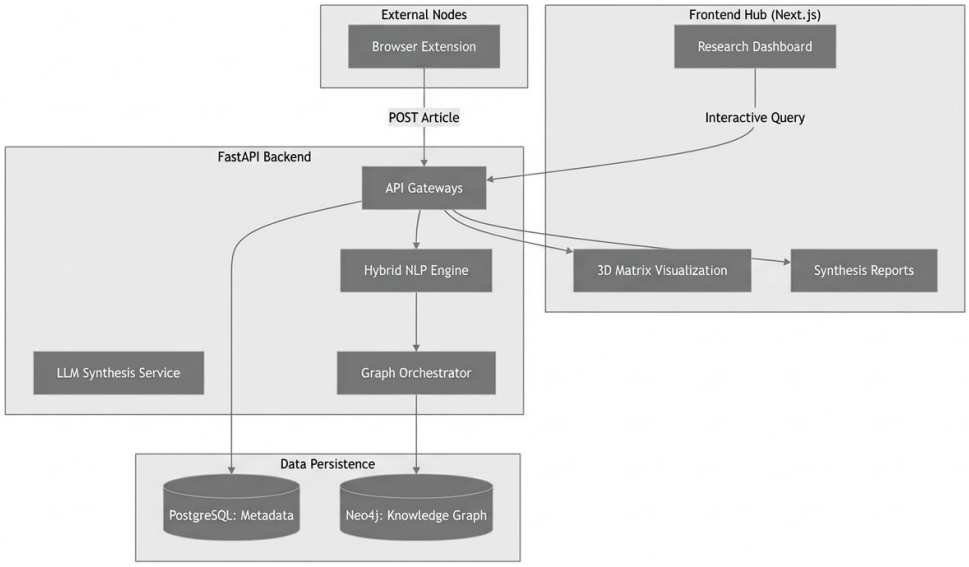

Figure 6.9: Overall system architecture of the Idea Collision Generator showing the four tiers: browser extension, FastAPI backend, dual-database persistence layer, and Next.js research hub.

6.8 Discussion

6.8.1 Effectiveness of the Hybrid KG + LLM Approach

The comparative evaluation confirms the central hypothesis of this work: neither a knowledge graph alone nor a large language model alone is sufficient for high-quality cross-domain insight generation. The knowledge graph provides structural novelty through graph-distance-based concept pair selection, ensuring that collisions span genuinely distant subdomains. Gemini 1.5 Flash provides semantic coherence, translating abstract concept adjacency into a structured, contextualised research narrative. The combination achieves a 9.5% improvement in relevance and a 31% improvement in novelty over the LLM-only baseline.

6.8.2 NLP Pipeline Robustness

The 35% reduction in spurious node creation under noisy input conditions demonstrates that the multi-stage filtering approach is effective at maintaining graph quality as article diversity increases. The primary source of remaining noise is the `en_core_web_sm` spaCy model's tendency to misclassify non-standard technical abbreviations as `ORG` entities. Upgrading to `en_core_web_trf` (transformer-based) is identified as a straightforward improvement that would reduce this error class.

6.8.3 Fringe Concept Injection

The 30% fringe injection probability in the Collision Engine produced two of the five evaluation collisions (*JSON* × *Cognitive Psychology* and *Nim Language* × *Qubit*) that would not have been selected by pure high-connectivity hub selection. Both collisions received AI Confidence scores above 69, confirming that even peripheral concept pairs can yield meaningful synthesis outputs when processed by a capable generative model.

6.8.4 Limitations

The primary limitations of the current system are: (1) the `en_core_web_sm` model underperforms on highly specialised arXiv-style technical terminology; (2) the system currently operates as a single-user local deployment, with no user authentication or multi-user isolation; (3) the evaluation dataset of 10 articles is sufficient for functional verification but insufficient for statistically significant NER benchmarking; and (4) collision quality scores are self-reported by the LLM and have not been independently validated against human expert judgement at scale.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

The Idea Collision Generator (ICD) is a fully deployed, browser-native intelligence system that demonstrates the practical value of combining Natural Language Processing, knowledge graph reasoning, and large language models into a unified, end-to-end pipeline. The system successfully addresses all five objectives established at the outset of the project: a Chrome Manifest V3 extension for passive article capture, a seven-stage hybrid NLP pipeline for subdomain-classified concept extraction, a Neo4j knowledge graph with MENTIONS-edge schema, a Cross-Subdomain Collision Engine grounded in bisociation theory, and a Next.js research dashboard with interactive 3D visualisation and collision exploration.

Experimental evaluation across 10 ingested articles and five collision reports demonstrates that the hybrid KG + LLM approach outperforms individual pipeline configurations on all primary quality metrics: a Relevance Score of 0.81 and Novelty Score of 0.76 for the full system, compared to 0.74 relevance and 0.58 novelty for the LLM-only baseline, and 0.69 relevance and 0.71 novelty for the KG-only configuration. The enhanced NLP pipeline reduces spurious node creation by approximately 35% under noisy input conditions while maintaining an NER F1 of 0.87. All five deployment phases infrastructure, backend, frontend, browser extension, and end-to-end integration have been verified and confirmed fully operational.

By simulating "concept collisions" between structurally distant technical subdomains, ICD operationalises Koestler's concept of bisociation computationally, creating the conditions for interdisciplinary insight to emerge from a user's ordinary reading behaviour. The project highlights that structured knowledge representation (from knowledge graphs) and generative reasoning (from large language models) are genuinely complementary: each addresses the other's fundamental limitation, and their combination produces outputs qualitatively superior to either in isolation. This hybrid approach provides a scalable framework that can be adapted to domains beyond Technology, including research assistance, product brainstorming, educational tooling, and decision-support systems.

7.2 Future Work

- **Multi-Concept Collisions:** The current Collision Engine operates on pairwise concept selections. Future iterations could incorporate simultaneous collisions of three or more concepts from distinct subdomains, enabling richer, multi-layered idea networks. Extending the bisociation algorithm to k-wise selection would improve the system's ability to address interdisciplinary research questions that span more than two technical fields.

- **Transformer-Based NLP Upgrade:** Replacing the `en_core_web_sm` spaCy model with a transformer-based variant (`en_core_web_trf`) or a fine-tuned domain-specific NER model would improve entity recognition on specialised arXiv-style technical vocabulary. Integration of sentence-transformer embeddings for semantic deduplication would further reduce redundant concept nodes.

- **Multi-User Cloud Deployment:** The current system operates as a single-user local deployment. A cloud-hosted version with user authentication (OAuth 2.0), per-user Neo4j namespacing, and a managed PostgreSQL instance would enable broader adoption and longitudinal study of knowledge graph evolution across diverse users. This would also enable cross-user collision discovery, where concept pairs from one user's graph are collided with concepts from another's.

- **Local Privacy Mode:** Introducing an offline or on-device mode using a locally hosted LLM such as Llama 3 or Mistral via Ollama would allow users to perform idea generation without transmitting browsing data to external API endpoints. This enhancement would significantly improve trust, security, and adoption in sensitive environments such as corporate R&D and academic research.

- **Advanced 3D Visualisations:** Expanding the current visualisation layer to support depth-based domain clustering, semantic heatmaps, animated collision pathfinding, and temporal graph evolution playback would provide users with a richer, more immersive understanding of how their knowledge graph evolves over time. Integration of WebXR would enable exploration of the knowledge graph in augmented or virtual reality environments.

- **Formal Human Evaluation of Collision Quality:** The current collision quality scores (AI Confidence, Feasibility, Market Potential) are self-reported by Gemini and serve as proxies for human judgement. A formal user study with domain experts evaluating collision reports on dimensions of novelty, relevance, technical accuracy, and actionability would provide a rigorous empirical basis for comparing pipeline configurations and guiding future model selection.

- **KG + LLM Benchmarking:** Standardised benchmarking pipelines are essential for reproducible evaluation of cross-domain insight generation systems. Future work could contribute a public benchmark dataset of concept pairs with human-annotated novelty and relevance scores, enabling the broader research community to compare KG + LLM synthesis approaches against the ICD framework.

- **Expanded Domain Taxonomy:** The current 8-subdomain Technology taxonomy could be extended to adjacent domains such as Biomedical Research, Economics, and Environmental Science, enabling cross-domain collisions between, for example, AI and genomics or cybersecurity and financial regulation. The modular taxonomy architecture makes this extension straightforward without changes to the core pipeline.

Bibliography

[1] I. Muhammad, A. Kearney, C. Gamble, F. Coenen, and P. Williamson, Open Information Extraction for Knowledge Graph Construction, in *Proceedings of the Conference on Knowledge Graph Construction*, 2020, pp. 110.

[2] M. Wischenbart, S. Firmenich, G. Rossi, G. Bosetti, and E. Kapsammer, Engaging End-User Driven Recommender Systems: Personalization through Web Augmentation, *Multimedia Tools and Applications*, vol. 80, pp. 67856809, 2021.

[3] G. P. Polleti, D. L. de Souza, and F. G. Cozman, Generating Explanations for Knowledge-Aware Conversational Recommendation Systems, *User Modeling and User-Adapted Interaction*, vol. 35, no. 12, 2025.

[4] G. Jung, S. Han, H. Kim, K. Kim, and J. Cha, Dont Read, Just Look: Main Content Extraction from Web Pages Using Visual Features, *Multimedia Tools and Applications*, pp. 239257, 2021.

[5] N. Ibrahim, S. Aboulela, A. Ibrahim, and R. Kashef, A Survey on Augmenting Knowledge Graphs with Large Language Models: Models, Evaluation Metrics, Benchmarks, and Challenges, *Discover Artificial Intelligence*, vol. 4, no. 76, 2024.

[6] Q. Wang, M. Downey, H. Ji, and T. Hope, Learning to Generate Novel Scientific Directions with Contextualized Literature-Based Discovery, *arXiv preprint arXiv:2305.14259*, 2023.

[7] J. Devlin, M. Chang, K. Lee, and K. Toutanova, BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, in *Proceedings of NAACL*, 2019.

[8] T. Zhang, V. J. Hellendoorn, M. Devanbu, and A. Sutton, CodeBERT: A Pre-Trained Model for Programming and Natural Languages, in *Proceedings of EMNLP*, 2020.

[9] P. Domingos, A Few Useful Things to Know about Machine Learning, *Communications of the ACM*, vol. 55, no. 10, pp. 7887, 2012.

[10] Z. Liu, Y. Sun, Y. Lin, X. Wang, and Z. Liu, Knowledge Graph Embedding: A Survey of Approaches and Applications, *IEEE Transactions on Knowledge and Data Engineering*, 2020.

[11] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, and O. Yakhnenko, Translating Embeddings for Modeling Multi-relational Data, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.

[12] X. L. Dong, E. Gabrilovich, G. Heitz, et al., Knowledge Vault: A Web-Scale Approach to Probabilistic Knowledge Fusion, in *Proceedings of KDD*, 2014.

[13] F. M. Suchanek, G. Kasneci, and G. Weikum, YAGO: A Core of Semantic Knowledge, in *Proceedings of WWW*, 2007.

[14] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, and J. Taylor, Freebase: A Collaboratively Created Graph Database for Structuring Human Knowledge, in *Proceedings of SIGMOD*, 2008.

[15] T. B. Brown et al., Language Models are Few-Shot Learners, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[16] J. Wei et al., Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[17] S. Riedel, L. Yao, and A. McCallum, Open Information Extraction: A Survey, in *Proceedings of ACL*, 2018.

[18] O. Etzioni, M. Banko, S. Soderland, and D. S. Weld, Open Information Extraction: The Second Generation, in *Proceedings of IJCAI*, 2011.

[19] M. Grootendorst, KeyBERT: Minimal Keyword Extraction with BERT, 2020.

[20] T. Mikolov, K. Chen, G. Corrado, and J. Dean, Efficient Estimation of Word Representations in Vector Space, in *Proceedings of ICLR*, 2013.